

Zadání projektu do předmětu IZG.

Table of Contents

Zadání projektu do předmětu IZG.

Jak na projekt?

Implementace funkce gpu_execute

1. Úkol - naprogramovat obsluhu command bufferu a čistění framebuffer

2. Úkol - Kreslení - počást vertex assembly jednotka a pouštění vertex shaderu

0. Vyvolávání vertex shaderu

1. Číslování kreslicích příkazů

2. Číslování vrcholů

3. Číslování vrcholů s indexováním.

4. Vertex shader by měl dostat konstanty z paměti.

5. Vertex Atributy - Vertex Assembly jednotka

3. Úkol - naprogramovat Primitive Assembly jednotku, rasterizaci a pouštění fragment shaderu

Primitive Assembly

Perspektivní dělení

Viewport transformace

Culling / Backface Culling

Rasterizace

Fragment processor

10. Ověření, že rasterizace produkuje fragmenty

11. Ověření, zda počítáte perspektivní dělení.

12. Ověření, zda vyrasterizované fragmenty mají správnou 2D pozici.

13. Ověření, zda se správně interpoluje hloubka fragmentů.

14., 15. Ověření, zda se správně interpolují vertex atributy.

4 Úkol - naprogramovat per fragment operace a zápis do framebufferu.

Hloubkový test

Blending

Blending + Depth modifikace

16. - 20. Ověření, zda se správně fungují per fragment operace

5. Úkol - naprogramovat ořez trojúhelníků blízkou ořezovou rovinou

Clipping

6. Úkol - Vykreslování modelů - funkce prepareModel

Průchod modelem

Paměť

7. Úkol - Vykreslování modelů - vertex shader drawModel_vertexShader

8. Úkol - Vykreslování modelů - fragment shader drawMode_fragmentShader

9. Úkol - finální render

Rozdělení souborů a složek

Sestavení

Spouštění

Ovládání

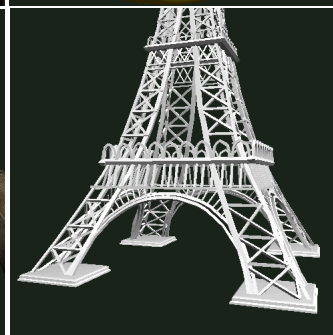
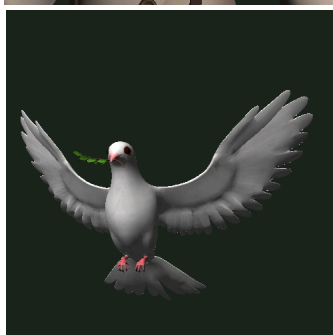
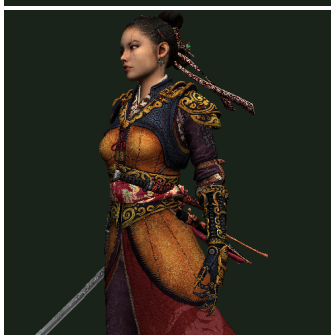
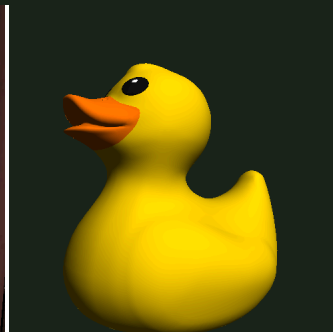
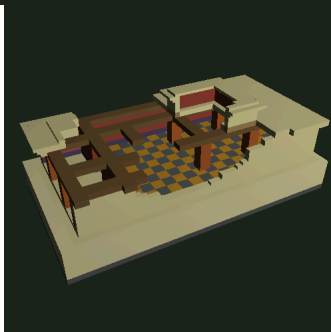
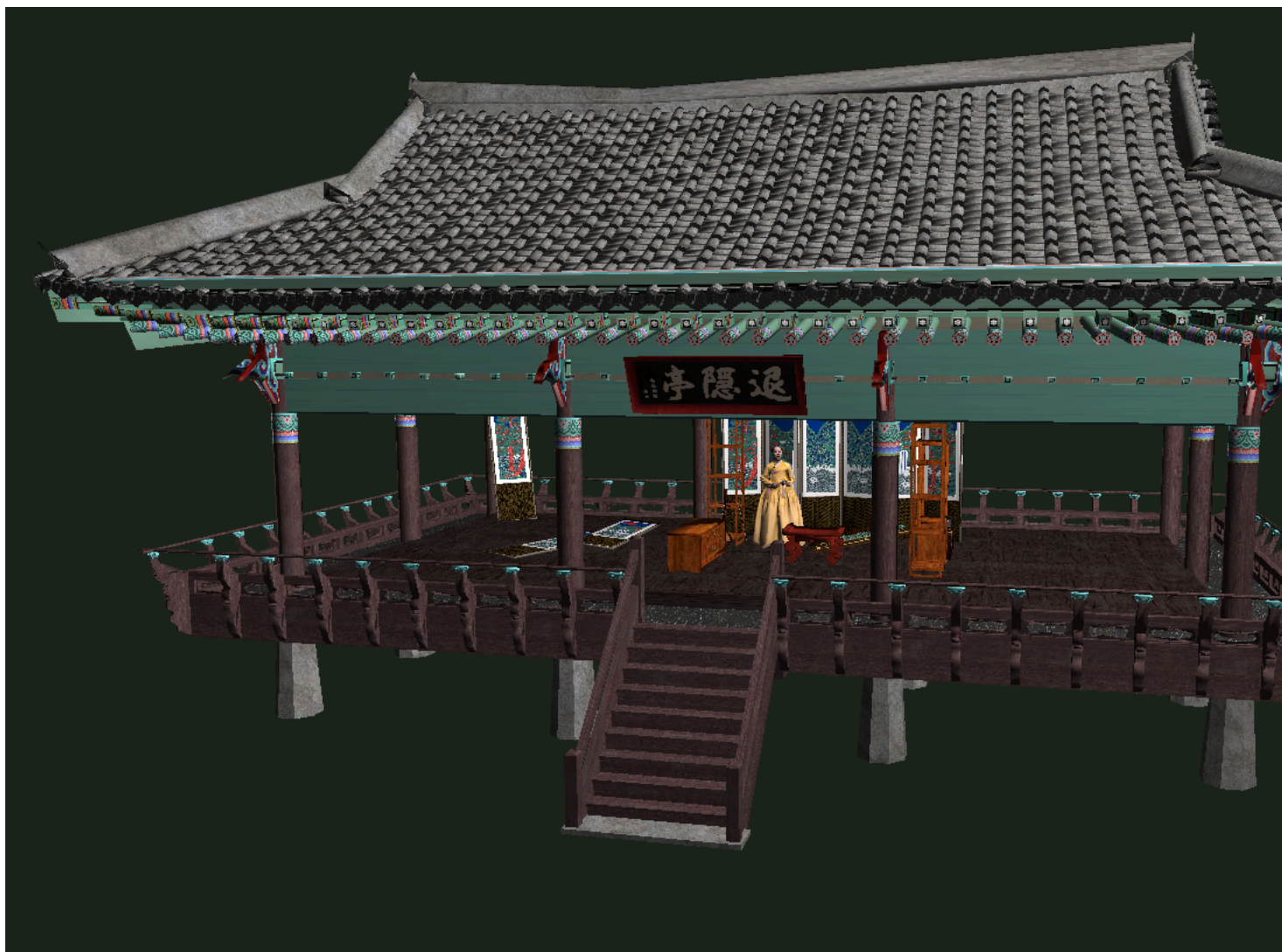
Testování

Odevzdávání

Časté chyby, které nedělejte

Hodnocení

Soutěž





Takto by měl vypadat výstup projektu

Vaším úkolem je naimplementovat jednoduchou grafickou kartu (gpu). A dále implementovat funkci pro vykreslení modelů. Všechny soubory, které se vás týkají jsou ve složce student/. V souboru `student/gpu.cpp` implementujte funkci `gpu_execute` - funkcionalita vámi implementované grafické karty. V souboru `student/drawModel.cpp` implementujete funkce `prepareModel`, `drawModel_vertexShader` a `drawModel_fragmentShader`. Tyto funkce slouží pro zpracování načteného souboru s modelem do paměti grafické karty a command bufferu. Kromě toho se ve složce nachází ještě soubor `student/fwd.hpp` - ten obsahuje deklarace struktur a konstant.

Jak na projekt?

Projekt se může zdát z prvu obrovský s milionem souborů a všelijakých podivností. Tyto "podivnosti" ale nemusíte řešit. Vše, co se vás týká jsou v podstatě 2 soubory do kterých napíšete váš kód a jeden soubor s deklaracemi struktur pro referenci. Projekt okolo těchto souborů vypadá takto z mnoha důvodů (vytvoření okna, načítání modelů, testování, ...). A není potřeba se jim zabývat (tedy pokud nechcete vidět vnitřnosti a jak celý projekt funguje). Takže jak na to?

Nejprve si vyzkoušejte, jak by to mělo vypadat...

```
# a mackejte "n" nebo "p" a ovladani mysí
izgProject_windows.exe
izgProject_windows.exe --method 13
izgProject_windows.exe --method 13 --model resources\\models\\thebes_palace\\scene.gltf
izgProject_windows.exe --method 13 --model resources\\models\\tf2\\scene.gltf
izgProject_windows.exe --method 13 --model resources\\models\\china\\china.glb
izgProject_windows.exe --method 13 --model resources\\models\\glorious_duck\\scene.gltf
izgProject_windows.exe --method 13 --model resources\\models\\nyra\\scene.gltf
izgProject_windows.exe --method 13 --model resources\\models\\peace\\scene.gltf
izgProject_windows.exe --method 13 --model resources\\models\\triss\\scene.gltf
izgProject_windows.exe --method 13 --model resources\\models\\eifel\\scene.gltf

# a mackejte "n" nebo "p" a ovladani mysí
./izgProject_linux.bin
./izgProject_linux.bin --method 13
./izgProject_linux.bin --method 13 --model resources/models/thebes_palace/scene.gltf
./izgProject_linux.bin --method 13 --model resources/models/tf2/scene.gltf
./izgProject_linux.bin --method 13 --model resources/models/china/china.glb
./izgProject_linux.bin --method 13 --model resources/models/glorious_duck/scene.gltf
./izgProject_linux.bin --method 13 --model resources/models/nyra/scene.gltf
./izgProject_linux.bin --method 13 --model resources/models/peace/scene.gltf
./izgProject_linux.bin --method 13 --model resources/models/triss/scene.gltf
./izgProject_linux.bin --method 13 --model resources/models/eifel/scene.gltf
```

Jak je to složité? Můj kód pro draw má ~400 řádků a implementace prepareModel a shaderů ~100 řádků. 500 řádků není moc... Není potřeba nic alokovat, paměť je již předchystaná. Takže pokud budete někde volat malloc, new a podobně, zamyslete se. Z C++ se nevyužívá skoro nic (jen vector a knihovna glm, reference). Takže by to mělo jít napsat celkem v pohodě i pro C lidi.

1. Vyzkoušet si přiložený zkompilovaný referenční projekt `izgProject_linux.bin` a `izgProject_windows.exe`. (mačkejte "n" nebo "p", když projekt pustíte, abyste přepínali zobrazované metody).
2. **Zprovoznit si překlad**
3. **Zkusit si projekt pustit a podívat se na parametry příkazové řádky, a jak se aplikace ovládá**
4. V projektu jsou přítomny **akceptační testy**, které vám řeknou, jestli jede správným směrem a taky vypisují napovědu.
5. Začít implementovat funkci `gpu_execute` a kontrolovat váš postup podle přiložených testů.
6. Začít implementovat funkci `prepareModel`
7. Začít implementovat funkci `drawModel_vertexShader`
8. Začít implementovat funkci `drawModel_fragmentShader`
9. Ověřte si implementaci na merlinovi
10. **Odevzdávání** Odevzdejte
11. ???
12. profit

Každý úkol má přiřazen akceptační test, takže si můžete snadno ověřit funkčnosti vaší implementace.

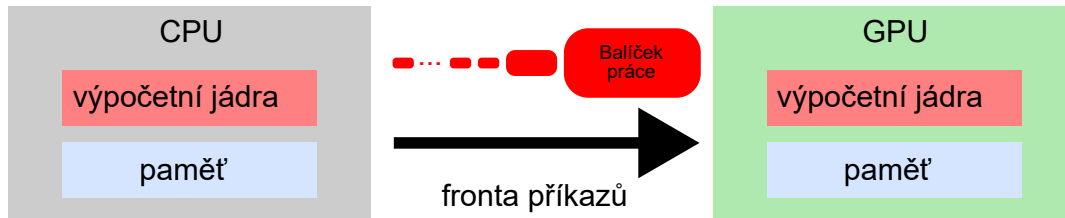
Implementace funkce `gpu_execute`

Funkce `gpu_execute` se nachází v `student/gpu.cpp`. Je to funkce, která reprezentuje chování grafické karty. Lze pomocí ní kreslit trojúhelníky nebo mazat framebuffer.

```
void gpu_execute(GPUMemory&mem,CommandBuffer &cb){
    (void)mem;
    (void)cb;
}
```

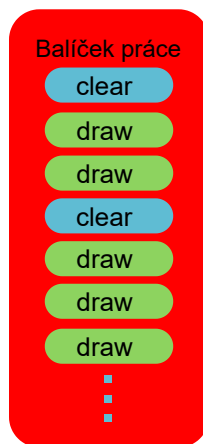
Vášim úkolem je ji postupně naprogramovat. Na jeden pokus ji nenaprogramujete, budete ji programovat postupně. Doporučuji si kousky funkce dávat do vlastních podfunkcí, ať máte kód přehledný.

První věc, na co se asi ptáte: "Jak vypadá počítač, grafická karta, jak se s ní komunikuje a co je její chování?"



Přehled toho, jak vypadá počítač. S grafickou kartou se komunikuje pomocí front příkazů, po které se posílají balíčky práce - Command Buffery.

Jak je vidět, tak s grafickou kartou, se komunikuje pomocí fronty příkazů (v tomto projektu není), po které se posílají balíčky práce. Balíček práce (**CommandBuffer**) v sobě obsahuje mnoho úkolů, které má grafická karta provést.



Balíček práce - Command Buffer. Command buffer obsahuje příkazy pro grafickou kartu. Může jich být mnoho, třeba pro vykreslení celého modelu.

Takto je struktura command bufferu definována v souboru `student/fwd.hpp`:

```
struct CommandBuffer{
    uint32_t static const maxCommands      = 10000;
    uint32_t static const nofCommands      = 0    ;
    Command commands[maxCommands]         ;
};
```

Jak můžete vidět, obsahuje tři položky: maximální počet příkazů, který může být uložen, počet uložených příkazů a samotné příkazy.

Naše grafická karta umožňuje jen dva druhy práce: čištění framebufferu a kreslení do framebufferu. Struktura samotného příkazu vypadá takto:

```
struct Command{
    CommandData data ;
    CommandType type = CommandType::EMPTY;
};
```

Je složena z typu a dat. Typ je enum:

```
enum class CommandType{
    EMPTY,
    CLEAR,
    DRAW ,
};
```

A data je union:

```
union CommandData{
    CommandData():drawCommand(){}
    ClearCommand clearCommand;
    DrawCommand drawCommand ;
};
```

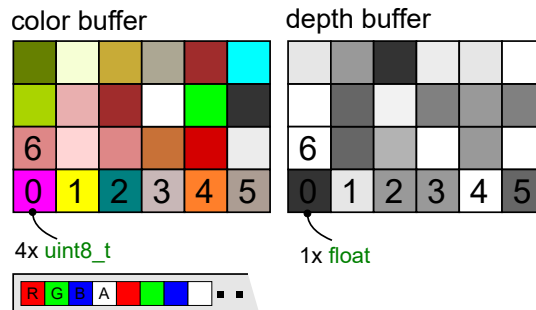
Union je něco jako struktura až na to, že jeho velikost je daná největší komponentou. Data unionu jsou uložena přes sebe a je možné uložit jen jednu komponentu. Vzhledem k tomu, že jsem projekt psal v C++, je přítomen i konstruktor, ale toho si nemusíte všimnout, jen udává, na co bude union inicializovaný - na draw command. Union obsahuje příkaz pro čištění framebuffer nebo kreslení.

1. Úkol - naprogramovat obsluhu command bufferu a čišťení framebuffer

Vášim prvním úkolem bude naprogramovat obsluhu command bufferu. Nejprve zprovozněte čišťení framebuffer (**Frame**). K tomuto úkolu se váže test 0 (budu uvádět jen Linuxové spouštění testů, na windows je to velmi obdobné):

```
./izgProject -c --test 0 --up-to-test
```

Framebuffer je složen ze dvou bufferu: paměť barvy (color buffer) a paměť hloubky (depth buffer):



Framebuffer - dva buffery: color buffer a depth buffer o stejném rozlišení. Framebuffer je plátno, kam se kreslí.

Oba dva mají stejné rozlišení. Barevný buffer má několik kanálů (4), každý po bajtu a hloubkový buffer má hloubku uloženou ve floatu. Framebuffer je koncipován tak, že pixel na souřadnicích [0,0] je v levém dolním rohu, osa X je doprava a osy Y nahoru.

Framebuffer se nachází v paměti grafické karty (**GPUMemory**):

```
struct GPUMemory{
    uint32_t const static maxUniforms = 10000;
    uint32_t const static maxTextures = 1000 ;
    uint32_t const static maxBuffers = 100 ;
    uint32_t const static maxPrograms = 100 ;
    Buffer buffers [maxBuffers ];
    Texture textures[maxTextures];
    Uniform uniforms[maxUniforms];
    Program programs[maxPrograms];
    Frame framebuffer;
};

struct Frame{
    uint8_t* color = nullptr;
    float * depth = nullptr;
    uint32_t channels = 4 ;
    uint32_t width = 0 ;
    uint32_t height = 0 ;
};
```

Čistící příkaz (**ClearCommand**) vypadá takto:

```
struct ClearCommand{
    glm::vec4 color = glm::vec4(0);
    float depth = 1e10 ;
    bool clearColor = true ;
    bool clearDepth = true ;
};
```

Čistící příkaz obsahuje barvu, hloubku, příznak čišťení barvy a příznak čišťení hloubky. Všimněte si, že barva je uložena jako floatový vektor glm::vec4. V tomto vektoru je barva v rozsahu [0,1] typu float. Čistící barvu musíte z toho rozsahu převést na rozsah [0,255] typu uint8_t.

Funkce **gpu_execute** má dva parametry: paměť a command buffer.

Takto vypadá pseudokód, jak můžete začít psát:

```
void clear(GPUMemory&mem, ClearCommand cmd){
    if(cmd.clearColor){
        float red = cmd.color.r;
        mem.framebuffer.color[0] = (uint8_t)red*255.f;
        //...
    }
}

void gpu_execute(GPUMemory&mem, CommandBuffer const&cb){
    for(uint32_t i=0; i<cb.nofCommands; ++i){
        CommandType type = cb.commands[i].type;
        CommandData data = cb.commands[i].data;
        if(type == CommandType::CLEAR){clear(mem, data.clearCommand);}
    }
}
```

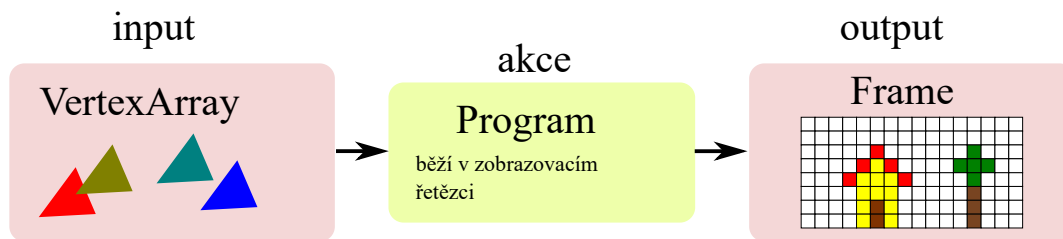
Dejte se do práce!

2. Úkol - Kreslení - počást vertex assembly jednotka a pouštění vertex shaderu

Druhý úkol je trochu delší. Kreslení je obtížnější a tak je rozloženo na podčásti. Nejprve trochu teorie.

Odkud se berou trojúhelníky a kam se kreslí?

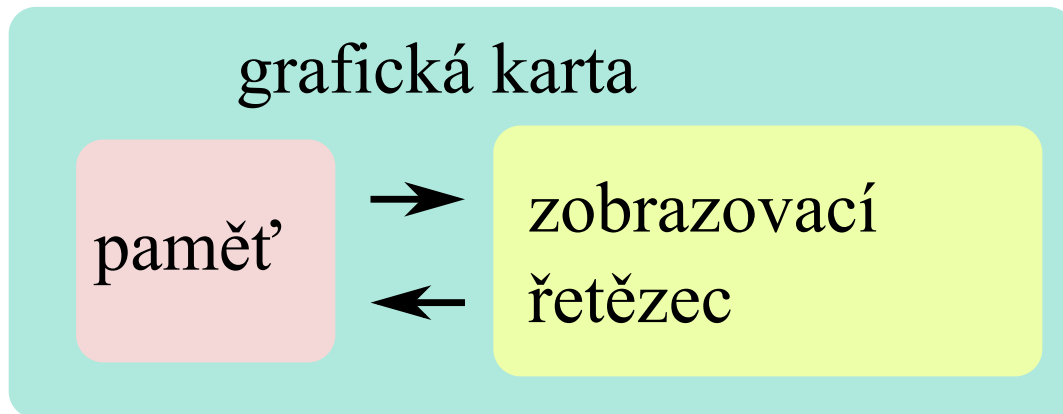
Trojúhelníky se čtou z paměti grafické karty, pak se rasterizují a výsledek se zapisuje zpátky do paměti. Akce/příkaz kreslení operuje nad paměti:



Přehled vykreslujícího příkazu. VertexArray je něco jako vstup pro vykreslování, jsou v něm 'zakódované' trojúhelníky. Frame je výstupní obrázek, který se kreslí. A program je společně s vykreslovacím řetězcem transformace vstupu na výstup.

Příkaz kreslení je prováděn stejně jako příkaz čistění v grafické kartě. Proces kreslení na grafické kartě probíhá v zobrazovacím řetězci.

Grafická karta

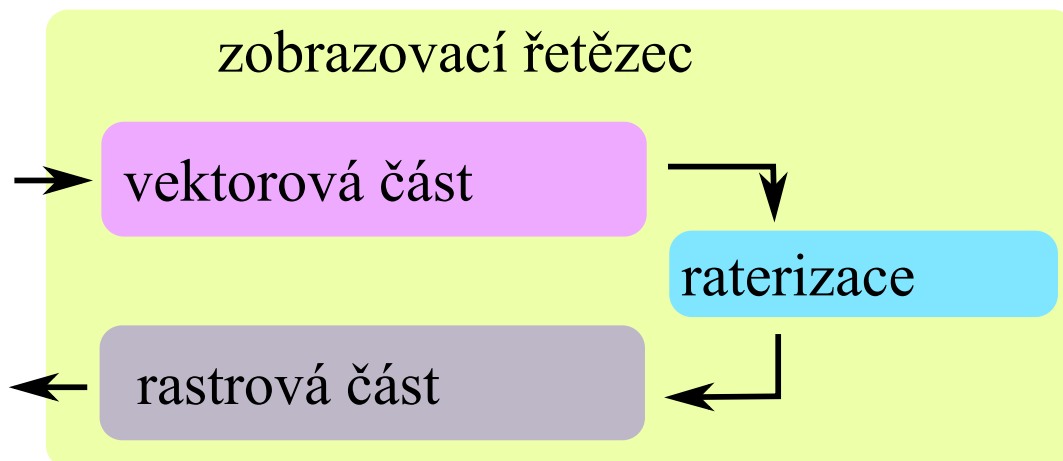


Grafická karta je složena z paměti a zobrazovacího řetězce. Z paměti tečou vrcholy a trojúhelníky, které jsou vyrasterizovány zpět do paměti.

Grafická provádí rasterizaci, hlavním účelem je převod vektorové grafiky na rastrovou. Zobrazovací řetězec je složitý, lze rozdělit na tři části: vektorová část, rasterizace a rastrová část.

Zobrazovací řetězec

Zobrazovací řetězec je složen ze tří částí: vektorová část, rasterizace, rastrová část.



Zobrazovací řetězec je složen z vektorové a rastrové části mezi kterými leží rasterizace.

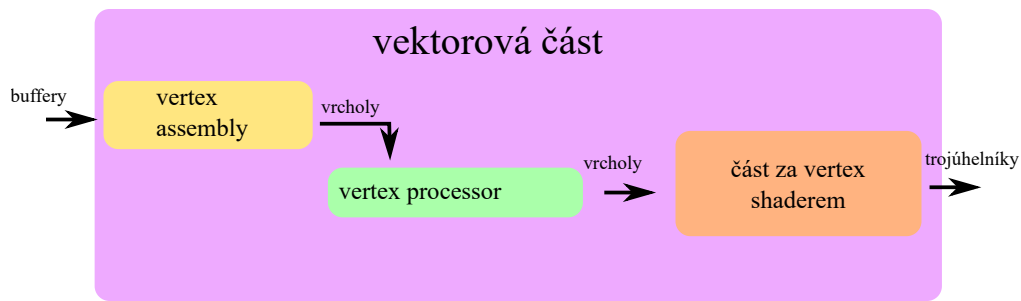
Úkolem vektorové části je transformovat vektorovou grafiku, posouvat trojúhelníky a podobně. Úkolem rasterizace je vektorové elementy převést na rastr. Úkolem rastrové části je obarvit vyrastrované vektory.

Část rasterizace a dál nás v tomto úkolu nezajímá, to až později. Tento test je zaměřený na vektorovou část a to jen na její vstup a vertex shader.

Vektorová část zobrazovací řetězce

Cílem vektorové části je zpravovávat vektorovou grafiku: body, trojúhelníky. Většinou se tím myslí: čtení z paměti a sestavení vrcholů, vyvolání vertex shaderu nad každým vrcholem, sestavení trojúhelníků, ořez, perspektivní dělení a připravení pro rasterizaci (viewport transformace). Rasterizace rasterizuje připravené trojúhelníky a produkuje fragmenty (čtvercové úlomky trojúhelníku, které se nakonec zapíší do framebufferu). Cílem rastrové části je obarvit tyto fragmenty pomocí fragment shaderu, odfiltrovat fragmenty, které jsou příliš daleko (depth test) a smíchat je s framebufferem (blending).

Ze začátku implementace kreslení se budete zabývat pouze vektorovou částí - a to částí před vertex shaderem (včetně).

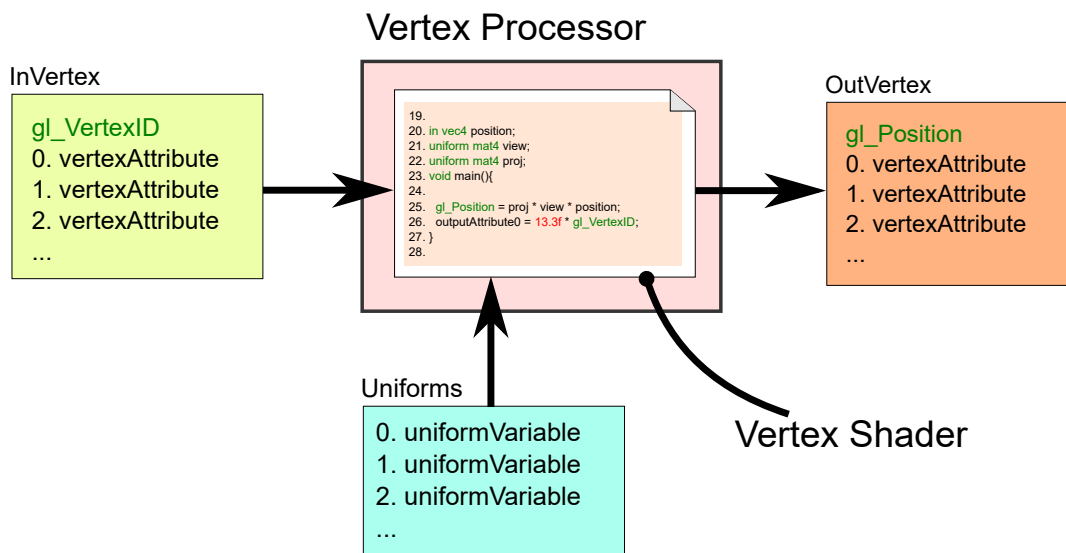


Vektorová část je složena z vertex assembly jednotky, vertex processoru a části za vertex shaderem.

Vertex assembly jednotka se stará o sestavování vrcholů. Vertex processor tyto vrcholy "prožene" uživatelem specifikovaným vertex shaderem. Část za vertex shaderem se stará o sestavení trojúhelníku, jeho ořezu a ztransformování pro rasterizaci.

Vertex Processor

Úkolem vertex processoru je pouštět uživatelem specifikovaný vertex shader. Obvykle provádí transformace vrcholů pomocí transformačních matic. Vertex processor vykonává shader (kus programu), kterému se říká vertex shader. Vstupem vertex shaderu je vrchol **InVertex**, výstupem je vrchol **OutVertex**. Dalším (konstatním) vstupem vertex shaderu jsou uniformní proměnné a textury **ShaderInterface**, které jsou uloženy v rámci shader programu. Pokud se uživatel rozhodne vykreslit 5 trojúhelníků je vertex shader spuštěn $5 \cdot 3 = 15$. Jednotlivé spuštění (invokace) vertex shaderu vyžadují nové vstupní vrcholy a produkují nové výstupní vrcholy. To ve výsledku znamená, že se pro každou invokaci vertex shaderu spustí Vertex Assembly jednotka, která sestaví vstupní vrchol.



Vizualizace vstupů a výstupů vertex procesoru. Ve vertex procesoru běží vertex shader, který obdrží vstupní vrchol, vyprodukuje výstupní vrchol a obdrží vstupní konstanty (uniformní proměnné a textury).

A teď práce. Úkol je zprovoznit vektorovou část zobrazovacího řetězce po rasterizaci. K tomu se vážou testy 1. - 9.

```
./izgProject -c --test 9 --up-to-test
```

Hrubý pseudokód může vypadat nějak takto:

```
void runVertexAssembly(){
    computeVertexID();
    readAttributes();
}

void draw(GPUMemory&mem,DrawCommand cmd){
    Program prg = mem.programs[cmd.programID];

    for(every vertex v < cmd.nofVertices){
        InVertex inVertex;
        OutVertex outVertex;
        runVertexAssembly(inVertex,cmd.vao,v);
        ShaderInterface si;
        prg.vertexShader(outVertex,inVertex,si);
    }
}

void gpu_execute(GPUMemory&mem,CommandBuffer const&cb){
    for(... commands ...){
        if (commandType == DRAW )
            draw(mem, drawCommand);
    }
}
```

I tak je to však složitý úkol, a tak je rozbitý na několik postupných testů.

0. Vyvolávání vertex shaderu

```
./izgProject -c --test 1
./izgProject -c --test 2
```

První test zkouší pouštět kreslení. Druhý míchá dohromady čistící a kreslicí příkazy.

Při kreslení musíte zavolat vertex shader tolikrát, kolik je zadáno v kreslicím příkazu (**DrawCommand**). Kreslicí příkaz je struktura:

```
struct DrawCommand{
    int32_t    programID      = -1  ;
    uint32_t   nofVertices    = 0   ;
    bool       backfaceCulling = false;
    VertexArray vao           ;
};
```

Struktura obsahuje počet vertexů pro vykreslení a číslo programu, který by se měl využít. Programy se nachází v paměti grafické karty **GPUMemory**.

```
struct GPUMemory{
    uint32_t const static maxUniforms = 10000;
    uint32_t const static maxTextures = 1000 ;
    uint32_t const static maxBuffers  = 100  ;
    uint32_t const static maxPrograms = 100  ;
    Buffer  buffers [maxBuffers ];
    Texture textures[maxTextures];
    Uniform uniforms[maxUniforms];
    Program programs[maxPrograms];
    Frame  framebuffer;
};
```

Program je opět struktura:

```
struct Program{
    VertexShader vertexShader = nullptr;
    FragmentShader fragmentShader = nullptr;
    AttributeType vs2fs[maxAttributes] = {AttributeType::EMPTY};
};
```

Struktura programu obsahuje vertex shader. Vertex shader je v ukazatel na funkci. Na normální GPU se jedná o program (třeba v GLSL), který se kompiluje. V projektu je to normální C/C++ funkce, která je uložena v ukazateli na funkci. Vertex shader bere 3 parametry

```
using VertexShader = void(*) (
    OutVertex      &outVertex,
    InVertex        const&inVertex ,
    ShaderInterface const&si       );
```

V tomto testu se neřeší, co dostane, ale měl by něco dostat.

1. Číslování kreslicích příkazů

```
./izgProject -c --test 3
```

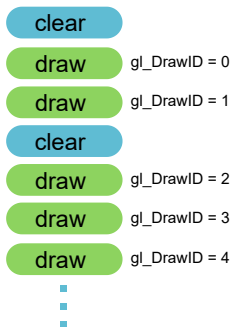
Jelikož může command buffer obsahovat vícero kreslicích příkazů, je obvykle nutné je číslovat. Toto číslování se používá pro výběr materiálů, textur, modelových matic a podobně. Číslicí se jen vykreslovací příkazy a to pomocí čísla `gl_DrawID`. Toto číslo součást struktury **InVertex**, což je vstup vertex shaderu:

```
struct InVertex{
    Attribute attributes[maxAttributes] = ;
    uint32_t   gl_VertexID              = 0;
    uint32_t   gl_DrawID                = 0;
};
```

ve které je položka **InVertex::gl_DrawID**, což je číslo vykreslovacího příkazu, které musíte správně nastavit. Kreslicí příkazy v command bufferu by měly dostat čísla 0,1,2,...

Pokud je mezi kreslicími příkazy čistící příkaz, neovlivní to číslování. Například, pokud je v command bufferu 5 příkazů: 3 kreslicí a 2 čistící, je jedno v jakém pořadí jsou, kreslicí dostanou čísla 0,1,2.

CommandBuffer



Číslování kreslicích příkazů.

2. Číslování vrcholů

```
./izgProject -c --test 4
```

Obdobně jako číslování kreslicích příkazů, existuje i číslování vrcholů. V tomto testu musíte správně číslovat vstupní vrcholy do vertex shaderu. Zatím bude stačit pořadové číslo. Vstupní vrchol se nachází ve struktuře **InVertex**

```
struct InVertex{
```



```

Attribute attributes[maxAttributes]
uint32_t gl_VertexID = 0;
uint32_t gl_DrawID = 0;
};

```

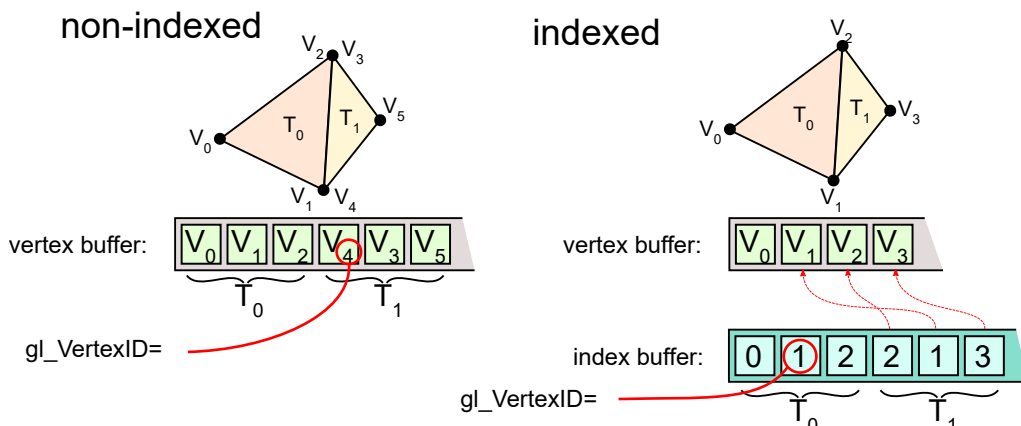
ve které je položka **InVertex::gl_VertexID**, což je číslo vrcholu, kterou musíte správně nastavit.

3. Číslování vrcholů s indexováním.

```
./izgProject -c --test 5
```

V tomto testu musíte správně číslovat vstupní vrcholy, když je zapnuté indexování.

Indexované kreslení je způsob snížení redundance dat s využitím indexů na vrcholy.



Neindexované a indexované kreslení.

U neindexovaného kreslení je číslo vrcholu **InVertex::gl_VertexID** rovno číslu invokace vertex shaderu. U indexovaného kreslení je číslo vrcholu **InVertex::gl_VertexID** rovno hodnotě z indexačního bufferu. Hodnota z indexačního bufferu je vybrána číslem invokace vertex shaderu.

Indexování může být zapnuto nebo vypnuto - o tom rozhoduje vykresovací příkaz:

```

struct DrawCommand{
    int32_t programID = -1 ;
    uint32_t nofVertices = 0 ;
    bool backfaceCulling = false;
    VertexArray vao ;
};

```

Ve vykreslovacím příkazu je položka **DrawCommand::vao**, což je tabulka nastavení takzvané vertex assembly jednotky. Toto nastavení je rovněž uloženo ve struktuře:

```

struct VertexArray{
    VertexAttrib vertexAttrib[maxAttributes];
    int32_t indexBufferID = -1;
    uint64_t indexOffset = 0 ;
    IndexType indexType = IndexType::UINT32;
};

```

V této struktuře jsou pro indexování podstatné položky **VertexArray::indexBufferID**, **VertexArray::indexOffset** a **VertexArray::indexType**. **indexBufferID** je číslo bufferu nebo -1 pokud je indexing vypnutý. **indexOffset** je posun v bajtech od začátku bufferu, kde se nacházejí indexy. **indexType** je typ indexu.

Všechny buffery (stejně jako programy) se nachází v paměti grafické karty (**GPUMemory**).

```

struct GPUMemory{
    uint32_t const static maxUniforms = 10000;
    uint32_t const static maxTextures = 1000 ;
    uint32_t const static maxBuffers = 100 ;
    uint32_t const static maxPrograms = 100 ;
    Buffer buffers [maxBuffers];
    Texture textures[maxTextures];
    Uniform uniforms[maxUniforms];
    Program programs[maxPrograms];
    Frame framebuffer;
};

```

Buffer je lineární paměť, reprezentovano strukturou:

```

struct Buffer{
    void const* data = nullptr;
    uint64_t size = 0 ;
};

```

Indexační buffer může mít různou velikost indexu - 8bit, 16bit a 32bit:

```

enum class IndexType{
    UINT8 = 1,
    UINT16 = 2,
    UINT32 = 4,
};

```

Pokud je zapnuto indexování, pak je číslo vrcholu dáno položkou v indexačním bufferu, kde je položka (index) v bufferu vybrána na základě čísla invokace vertex shaderu.

4. Vertex shader by měl dostat konstanty z paměti.

```
./izgProject -c --test 6
```

Uniformní proměnné a textury jsou z pohledu vertex shaderu konstanty. Jsou to data, která uživatel dodal svému vertex shaderu. Obvykle jsou to matice, pozice světla, barva materiálu... Všechny uniformní proměnné a textury jsou uloženy v paměti grafické karty (**GPUMemory**).

```
struct GPUMemory{
    uint32_t const static maxUniforms = 10000;
    uint32_t const static maxTextures = 1000 ;
    uint32_t const static maxBuffers = 100 ;
    uint32_t const static maxPrograms = 100 ;
    Buffer buffers [maxBuffers ];
    Texture textures[maxTextures];
    Uniform uniforms[maxUniforms];
    Program programs[maxPrograms];
    Frame framebuffer;
};
```

Vertex Shader má kromě vrcholů **InVertex** a **OutVertex** i vstup **ShaderInterface**

```
using VertexShader = void(*)(
    OutVertex      &outVertex,
    InVertex       const&inVertex ,
    ShaderInterface const&si );

struct ShaderInterface{
    Uniform const*uniforms = nullptr;
    Texture const*textures = nullptr;
};
```

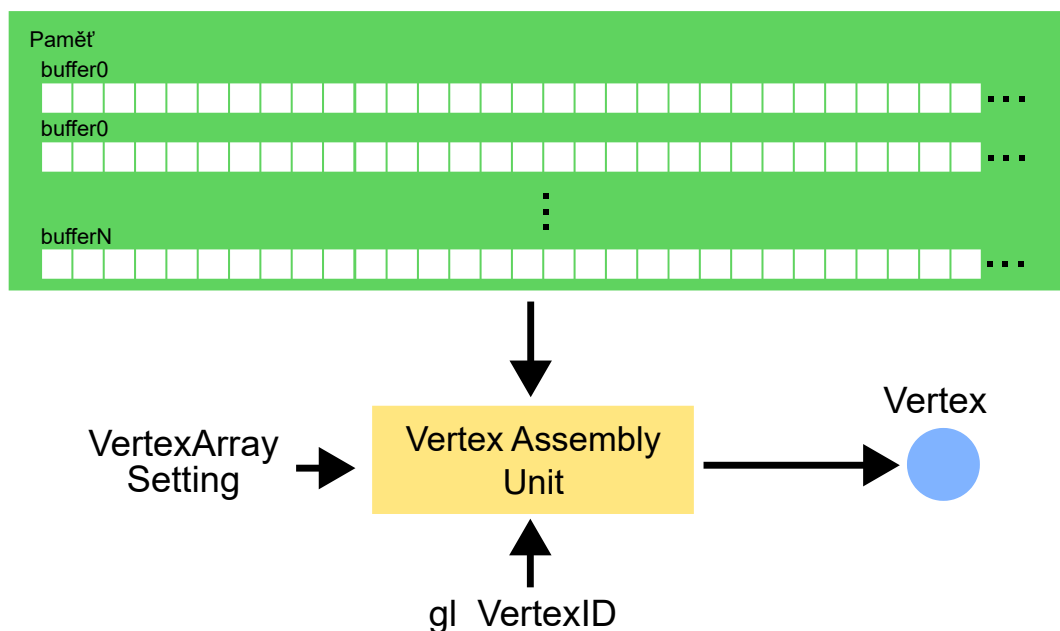
Struktura **ShaderInterface** jednoduše odkazuje na tabulky uniformních proměnných a textur.

5. Vertex Atributy - Vertex Assembly jednotka

```
./izgProject -c --test 7
./izgProject -c --test 8
./izgProject -c --test 9
```

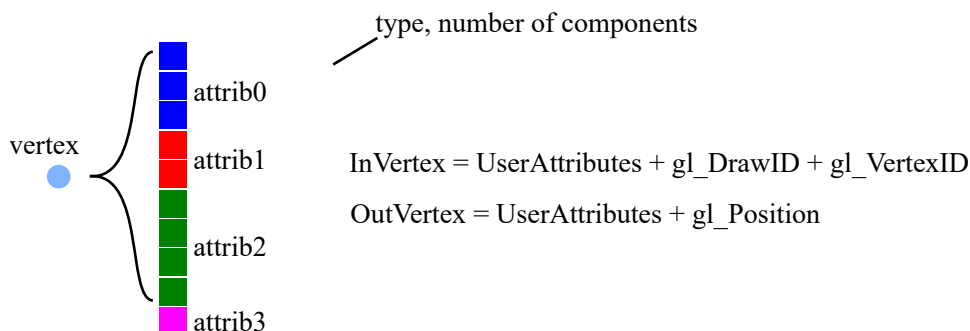
V tomto testu musíte naprogramovat funkcionalitu Vertex Assembly jednotky.

Vertex Assembly (nebo taky Vertex Puller, Vertex Specification, ...) je zařízení na grafické kartě, které se stará o sestavení vrcholů.



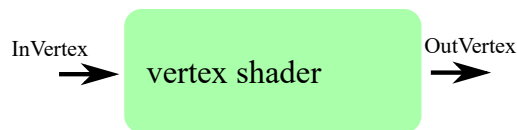
Vertex Assembly jednotka sestavuje vrcholy z paměti (většinou). K tomu potřebuje tabulka nastavení (VertexArray) a číslo vrcholu.

Vertex není jen bod v prostoru. Vertex je uživatelem specifikovaná struktura. Uživatel může mít po vrcholech různé požadavky a tak do vrcholů může přidat různé množství atributů – vertex atributů. Kromě uživatelem specifikovaných atributů, obsahují i pevně vestavěné atributy (`gl_DrawID`, `gl_VertexID` a další).



Vrchol je struktura dat. Ve struktuře jsou vertex atributy. Tyto mají uživatelem specifikovaný význam (třeba pozice, normála, ...). Vertex Assembly sestavuje **InVertex**, který je poslán jako vstup do vertex shaderu. **OutVertex** je výstup vertex shaderu.

Sestavené vrcholy jsou posílány do vertex shaderu pro zpracování uživatelem definovaným kódem. Vertex shader transformuje vrcholy maticemi a vypočítává výstupní vrcholy.



Jsou dva typy vrcholů. Ty, které sestavuje jednotka Vertex Assembly a vstupují do vertex shaderu. A ty, které jsou výstupem vertex shaderu.

Vrchol (**InVertex**) je složen z **maxAttributes** vertex atributů, každý může být různého typu (**AttributeType** (float, vec2, vec3, vec4, ...), čísla vykreslovacího příkazu **InVertex::gl_DrawID** a čísla vrcholu **InVertex::gl_VertexID**.

Struktura **InVertex** vypadá takto:

```

struct InVertex{
    Attribute attributes[maxAttributes] = ;
    uint32_t gl_VertexID = 0;
    uint32_t gl_DrawID = 0;
};
  
```

Struktura **OutVertex** vypadá takto:

```

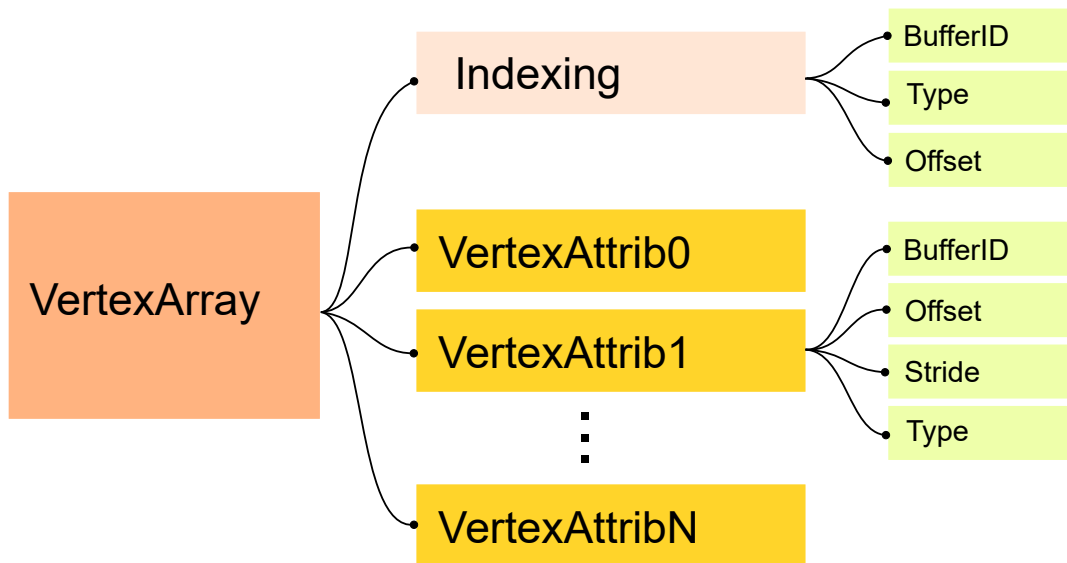
struct OutVertex{
    Attribute attributes[maxAttributes] = ;
    glm::vec4 gl_Position = glm::vec4(0,0,0,1);
};
  
```

Obě struktury obsahují vertex atributy **Attribute**

```

union Attribute{
    Attribute():v4(glm::vec4(1.f)){}
    float v1;
    glm::vec2 v2;
    glm::vec3 v3;
    glm::vec4 v4;
    uint32_t u1;
    glm::uvec2 u2;
    glm::uvec3 u3;
    glm::uvec4 u4;
};
  
```

Vertex Assembly jednotka se řídí podle nastavení ze struktury **VertexArray**.



Nastavení pro Vertex Assembly jednotku - VertexArray.

Toto nastavení je uloženo ve struktuře **VertexArray**:

```

struct VertexArray{
    VertexAttrib vertexAttrib[maxAttributes];
    int32_t indexBufferID = -1;
    uint64_t indexOffset = 0;
    IndexType indexType = IndexType::UINT32;
};
  
```

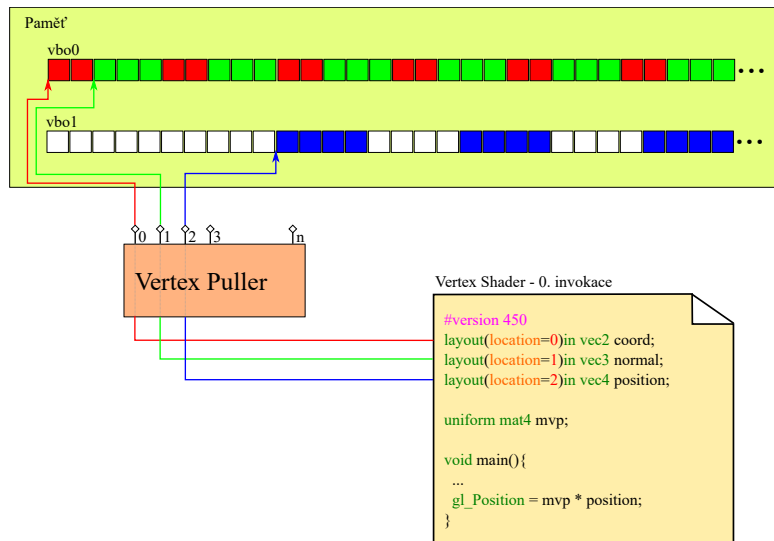
Je složeno z nastavení pro indexování a nastavení pro vertex atributy. **VertexAttrib** je struktura obsahující nastavení, jak číst jeden Vertex Atribut.

```

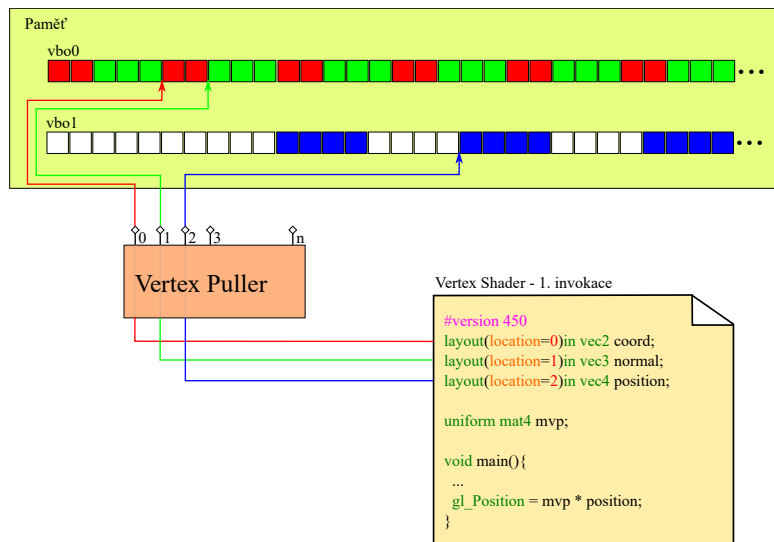
struct VertexAttrib{
    int32_t bufferID = -1;
    uint64_t stride = 0;
    uint64_t offset = 0;
    AttributeType type = AttributeType::EMPTY;
};
  
```

Vertex Assembly jednotka je složena z **maxAttributes** čtecích hlav, které sestavují jednotlivé vertex atributy. **InVertex** je složen z **maxAttributes** atributů, každý odpovídá jedné čtecí hlavě z Vertex Assembly jednotky. Čtecí hlava obsahuje nastavení - offset, stride, type a buffer. Pokud je čtecí hlava povolena (typ není empty), měla by zkopírovat data (o velikosti vertex atributu) z bufferu od daného offsetu, s krokem stride. Všechny velikosti jsou v bajtech. Krok se použije při čtení různých vrcholů: atributy by měly být čteny z adresy: `buf_ptr + offset + stride*gl_VertexID`

Na dalších dvou obrázcích je příklad stavu Vertex Assembly jednotky ve dvou (0. a 1.) invokaci vertex shaderu.



příklad vertex pulleru při 0. invokaci vertex shaderu. Vertex je složen z 3 vertex atributů (coord, normal, position). Čtecí hlavy začínají na daných offsetech a v daných bufferech.



příklad vertex pulleru při 1. invokaci vertex shaderu. Čtecí hlavy se posunuly o krok (stride).

Po těchto úkolech byste měli mít hotovotou část před vertex shaderem.

3. Úkol - naprogramovat Primitive Assembly jednotku, rasterizaci a použití fragment shaderu

V tomto úkolu je potřeba rozšířit funkcionalitu funkce `gpu_execute` o schopnosti rasterizace. Cílem je naprogramovat části zobrazovacího řetězce, které jsou za vertex shaderem po rasterizaci a použití fragment shaderu (včetně).

Jedná se o testy 10. - 15.

Pseudokód může po upravení vypadat nějak takto:

```
void runVertexAssembly(){
    computeVertexID();
    readAttributes();
}

void runPrimitiveAssembly(primitive,VertexArray vao,t,Program prg){
    for(every vertex v in triangle){
        InVertex inVertex;
        runVertexAssembly(inVertex,vao,t,v);
        prg.vertexShader(primitive.vertex,inVertex,prg.uniforms);
    }
}

void rasterizeTriangle(frame,primitive,prg){
    for(pixels in frame){
        if(pixels in primitive){
            InFragment inFragment;
            createFragment(inFragment,primitive,barycentrics,pixelCoord,prg);
            OutFragment outFragment;
            prg.fragmentShader(outFragment,inFragment,uniforms);
        }
    }
}

void draw(GPUMemory&mem,DrawCommand const&cmd){
    for(every triangle t){
        Primitive primitive;
        runPrimitiveAssembly(primitive,ctx.vao,t,ctx.prg)

        runPerspectiveDivision(primitive)
    }
}
```

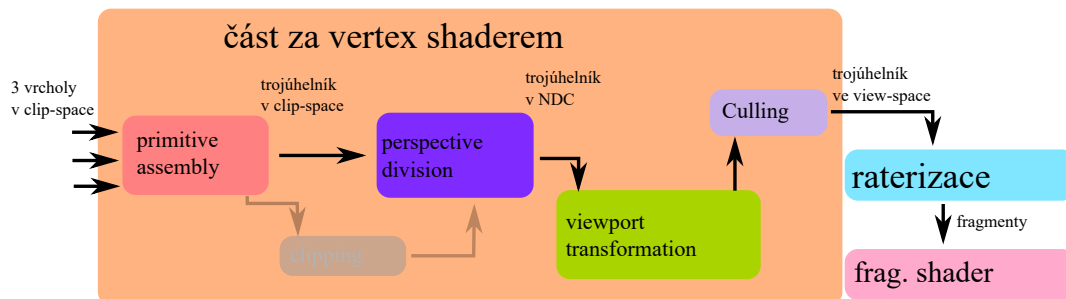
```

runViewportTransformation(primitive, ctx.frame)
rasterizeTriangle(ctx.frame, primitive, ctx.prg);
}
}

void gpu_execute(...){
}

```

Vertex Assembly jednotka chrlí vrcholy a vertex shader je zpracovává, transformuje. Je na čase z nich sestavit trojúhelníky a připravit je pro rasterizaci. Část za vertex shaderem je složena z několika částí.

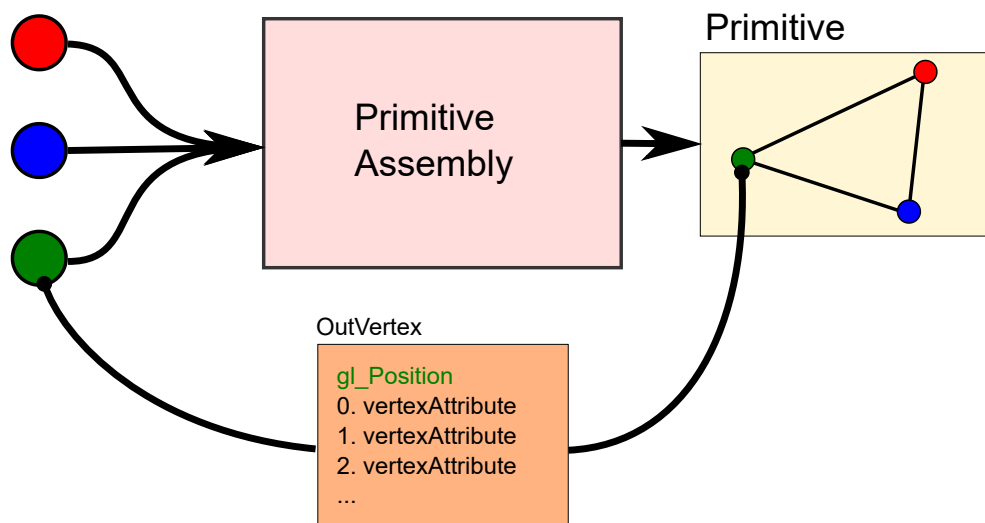


Část za vertex shaderem je složena z několika jednotek. Je to jednotka sestavení primitiv (trojúhelníků), ořez (ten teď dělat nebudete), perspektivní dělení, viewport transformace, culling. Po vektorové části následuje rasterizace a fragment shader

Primitive Assembly

Primitive Assembly je jednotka, která sestavuje trojúhelníky (mimo jiné). Trojúhelníky, úsečky, bodu se hromadně říká primitivum. V tomto projektu se používají pouze trojúhelníky. Primitive Assembly jednotka si počká na 3 po sobě jdoucí **výstupní vrcholy** z vertex shaderu a sestaví trojúhelník (struktura, která by měla obsahovat 3 výstupní vrcholy). Lze na to také nahlížet tak, že primitive assembly jednotka dostane příkaz vykreslit třeba 4 trojúhelníky. Jednotka tak spustí vertex shader 12x, který takto spustí 12x vertex assembly jednotku.

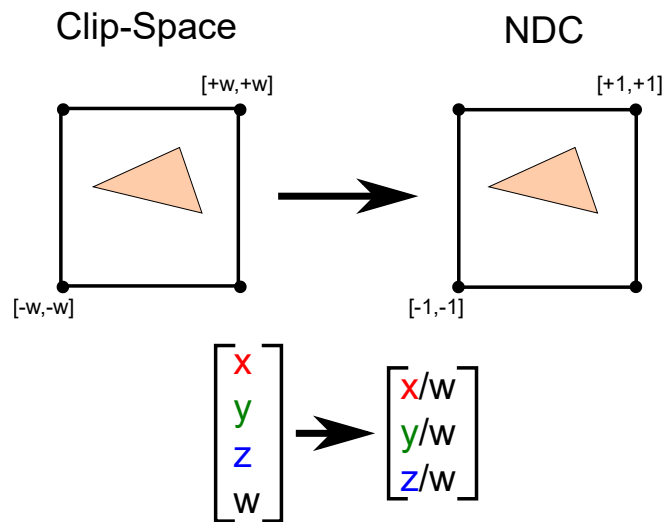
Out Vertices



Vizualizace funkce primitive assembly jednotky. Primitive assembly jednotka sestaví trojúhelník ze 3 po sobě jdoucích výstupních vrcholů z vertex shaderu.

Perspektivní dělení

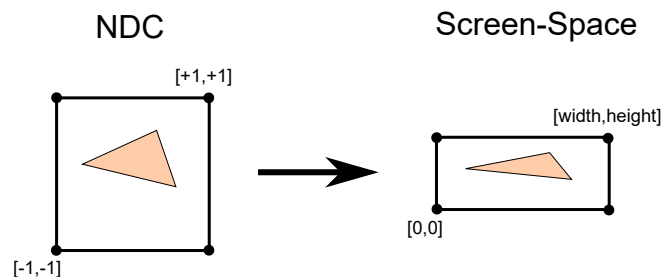
Perspektivní dělení následuje za clippingem (ten bude až později, teď není potřeba) a provádí převod z homogenních souřadnic na kartézské pomocí dělení w.



Perspektivní dělení. Převod z clip-space do NDC (normalized device coordinates). Dělí se pomocí w . Při tomto dělení vzniknou normalizované souřadnice x, y a normalizovaná hloubka.

Viewport transformace

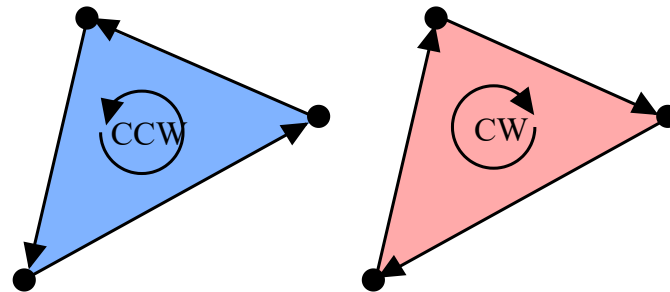
Viewport transformace provádí převod NDC (rozsah $-1, +1$) na rozlišení okna, aby se mohla provést rasterizace.



Vizualizace viewport transformace. Trojúhelníky jsou roztaženy na rozlišení obrazovky $[width, height]$ do screen-space. Hloubka zůstane zachována v komponentě z .

Culling / Backface Culling

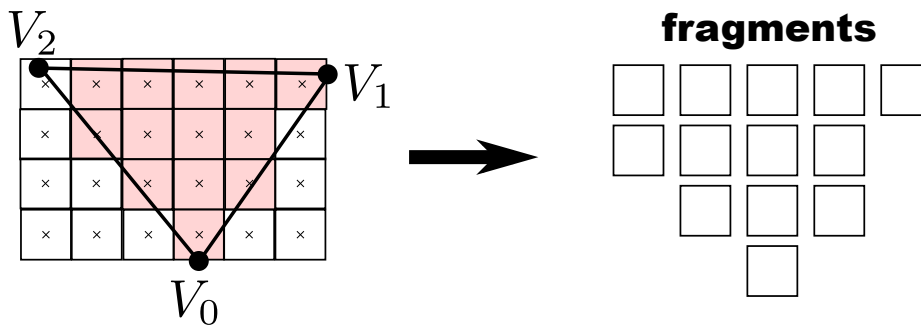
Backface Culling se stará o zahození trojúhelníků, které jsou odvráceny od pozorovatele. Culling lze zapnout nebo vypnout pomocí: `DrawCommand::backfaceCulling`. Pokud je zapnutý, trojúhelníky, které jsou specifikovány po směru hodinových ručiček jsou zahazovány. Pokud je vypnutý, vykreslují se všechny trojúhelníky - přivrácené i odvrácené - specifikované po i proti směru hodinových ručiček.



Levý trojúhelník je specifikovaný proti směru hodinových ručiček (counter clock wise) a je přivrácený k pozorovateli. Pravý je po směru (clock wise) a je odvrácený od uživatele. Levý se bude vždy vykreslovat, pravý pouze tehdy, pokud je back face culling vypnutý.

Rasterizace

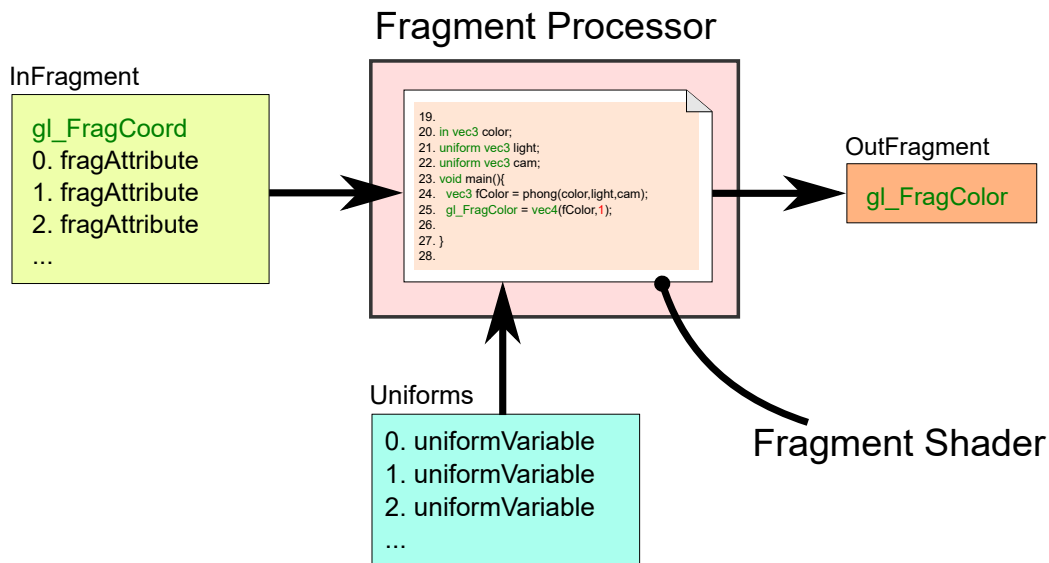
Rasterizace rasterizuje trojúhelník ve screen-space. Rasterizace produkuje fragmenty v případě, že **střed** pixelu leží uvnitř trojúhelníka.



Rasterizace produkuje fragmenty. Pokud střed pixelu leží uvnitř trojúhelníka, vytvoří se fragment.

Fragment processor

Fragment processor spouští fragment shader nad každým fragmentem. Data pro fragment shader jsou uložena ve struktuře **InFragment**. Výstup fragment shaderu je výstupní fragment **OutFragment** - barva. Další (konstantní) vstup fragment shaderu jsou uniformní proměnné a textura.



Vizualizace vstupů a výstupů fragment shaderu. Fragment Shader se používá nad každým vyrasterizovaným fragmentem.

10. Ověření, že rasterizace produkuje fragmenty

V tomto úkolu budete muset naprogramovat rasterizaci. Neobejdete se bez viewport transformace, rasterizace a zavolání fragment shaderu nad každým fragmentem. Tento test spočívá ve zkoušení vyrasterizování několika různých trojúhelníků a počítání, kolik se vyrasterizovalo fragmentů.

Testy pusťte:

```
izgProject -c --test 10
```

11. Ověření, zda počítáte perspektivní dělení.

Tento test ověřuje, zda provádíte perspektivní dělení.

```
izgProject -c --test 11
```

12. Ověření, zda vyrasterizované fragmenty mají správnou 2D pozici.

Tento test ověřuje, zda vyrasterizované fragmenty mají správnou 2D pozici **InFragment::gl_FragCoord**.

```
izgProject -c --test 12
```

Fragmenty mají souřadnice středů pixelů, kterým náleží. Tzn. fragment pro levý dolní pixel [0,0] má souřadnice **gl_FragCoord.xy** = [0.5,0.5]

13. Ověření, zda se správně interpoluje hloubka fragmentů.

Tento test ověřuje, zda vyrasterizované fragmenty mají správně interpolovanou hloubku.

```
izgProject -c --test 13
```

Hloubka fragmentu je komponentě "z" položky **InFragment::gl_FragCoord**. Pro její interpolaci potřebujete hloubky vrcholů trojúhelníka a barycentrické souřadnice fragmentu ve 2D.

Hloubky vrcholů najdete ve "z" komponentě položky **OutVertex::gl_Position** **gl_Position.z**

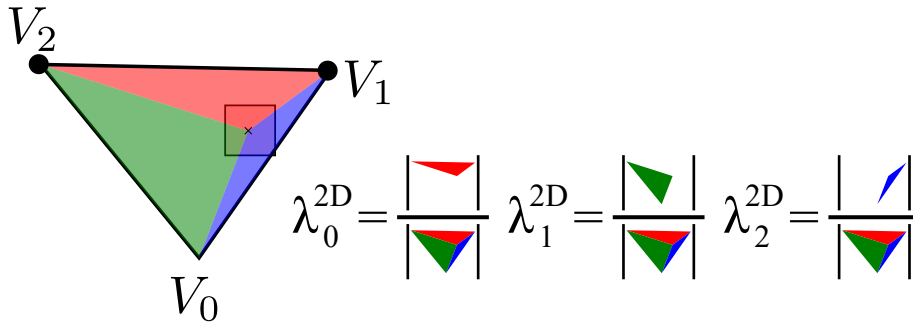
```
struct OutVertex{
```

```
Attribute attributes[maxAttributes]
glm::vec4 gl_Position          = glm::vec4(0,0,0,1);
};
```

Hloubku zapisujte do komponenty "z" položky `InFragment::gl_FragCoord` `gl_FragCoord.z`

```
struct InFragment{
    Attribute attributes[maxAttributes]
    glm::vec4 gl_FragCoord          = glm::vec4(1);
};
```

Barycentrické souřadnice musíte spočítat podle obsahů:



Barycentrické souřadnice ve 2D jsou spočítány jako poměry obsahů podtrojúhelníků.

Hloubka se interpoluje pomocí barycentrických souřadnic ve 2D:

$$fragment.gl_FragCoord.z = vertex[0].gl_Position.z \cdot \lambda_0^{2D} + vertex[1].gl_Position.z \cdot \lambda_1^{2D} + vertex[2].gl_Position.z \cdot \lambda_2^{2D}$$

Hloubka vrcholů `vertex[i].gl_Position.z` vznikla při perspektivním dělení.

14., 15. Ověření, zda se správně interpolují vertex atributy.

Tyto dva testy ověřují, jestli se správně interpolují vertex atributy do fragment atributů.

```
izgProject -c --test 14
izgProject -c --test 15
```

Vertex Atributy jsou se struktúře `OutVertex`

```
struct OutVertex{
    Attribute attributes[maxAttributes]
    glm::vec4 gl_Position          = glm::vec4(0,0,0,1);
};
```

A ze tří těchto vrcholů by se měly interpolovat atributy `InFragment`.

```
struct InFragment{
    Attribute attributes[maxAttributes]
    glm::vec4 gl_FragCoord          = glm::vec4(1);
};
```

Interpolujte pouze ty atributy, které jsou poznačené v položce `Program::vs2fs!` A pouze ty, které nejsou typu integer! Integerové atributy neinterpolujte, ale pouze použijte hodnoty nultého vrcholu. Tomuto vrcholu se také říká provoking vertex.

```
struct Program{
    VertexShader vertexShader = nullptr;
    FragmentShader fragmentShader = nullptr;
    AttributeType vs2fs[maxAttributes] = {AttributeType::EMPTY};
};
```

Atributy je potřeba interpolovat pomocí perspektivně korektně upravených barycentrických souřadnic. Perspektivní korektní interpolace:

$$\frac{\frac{A_0 \cdot \lambda_0^{2D}}{h_0} + \frac{A_1 \cdot \lambda_1^{2D}}{h_1} + \frac{A_2 \cdot \lambda_2^{2D}}{h_2}}{\frac{\lambda_0^{2D}}{h_0} + \frac{\lambda_1^{2D}}{h_1} + \frac{\lambda_2^{2D}}{h_2}}$$

Kde λ_0^{2D} , λ_1^{2D} , λ_2^{2D} jsou barycentrické koordináty ve 2D, h_0 , h_1 , h_2 je homogenní složka vrcholů a A_0 , A_1 , A_2 je atribut vrcholu.

Homogenní složka vrcholů je čtvrtá složka - tím čím se dělílo ve perspektivním dělení: $h_0 = vertex[0].gl_Position.w$, $h_1 = vertex[1].gl_Position.w$, ...

Barycentrické souřadnice je možné přepočítat na perspektivně korektní barycentrické souřadnice (je to jen přepsání zvorečku nahoře):

$$s = \frac{\lambda_0^{2D}}{h_0} + \frac{\lambda_1^{2D}}{h_1} + \frac{\lambda_2^{2D}}{h_2}$$

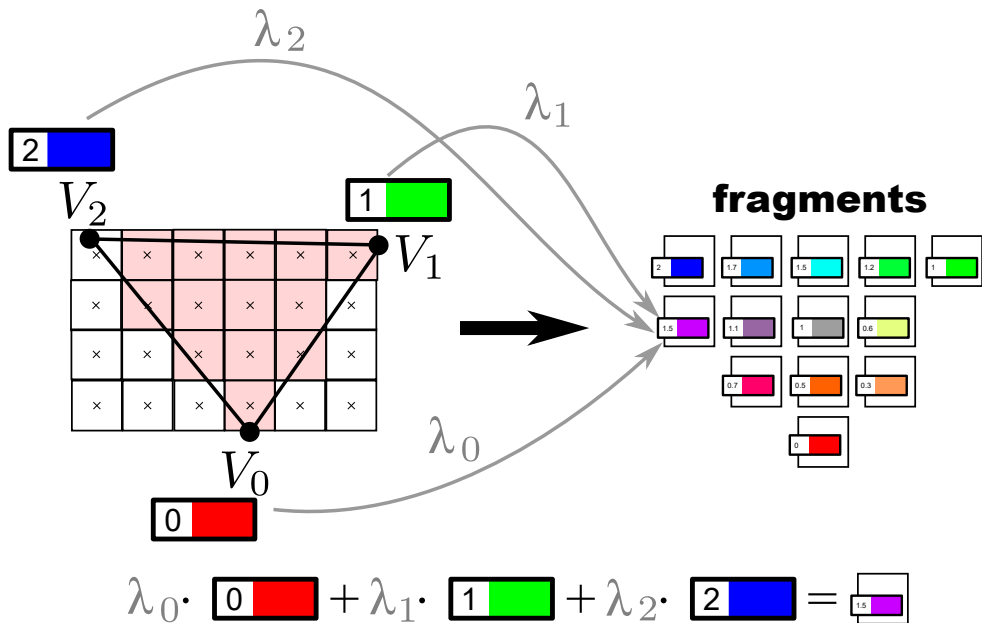
$$\lambda_0 = \frac{\lambda_0^{2D}}{h_0 \cdot s}$$

$$\lambda_1 = \frac{\lambda_1^{2D}}{h_1 \cdot s}$$

$$\lambda_2 = \frac{\lambda_2^{2D}}{h_2 \cdot s}$$

Ty je potom možné použít pro interpolaci atributů:

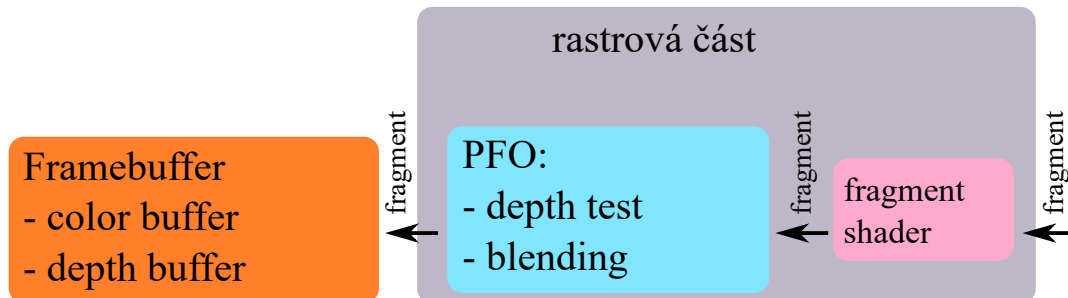
$$fragment.attribute = vertex[0].attribute \cdot \lambda_0 + vertex[1].attribute \cdot \lambda_1 + vertex[2].attribute \cdot \lambda_2$$



Rasterizace a interpolace vertex atributů. Vertex Atributy jsou interpolovány pomocí perspektivně korektních barycentrických souřadnic $\lambda_0, \lambda_1, \lambda_2$.

4 Úkol - naprogramovat per fragment operace a zápis do framebufferu.

Rastrovou část zobrazovacího řetězce už byla částečně nakousnutá v předcházejícím úkolu (byl volán fragment shader). Rastrová část vypadá takto:



Rastrová část je složena z fragment shaderu a per fragment operací. Jsou dvě PFO operace: hloubkový test a blending. Fragmenty se poté přimíchají do framebufferu.

PFO operují s výstupním fragmentem fragment shaderu `OutFragment`

```
struct OutFragment{
    glm::vec4 gl_FragColor = glm::vec4(0.f);
};
```

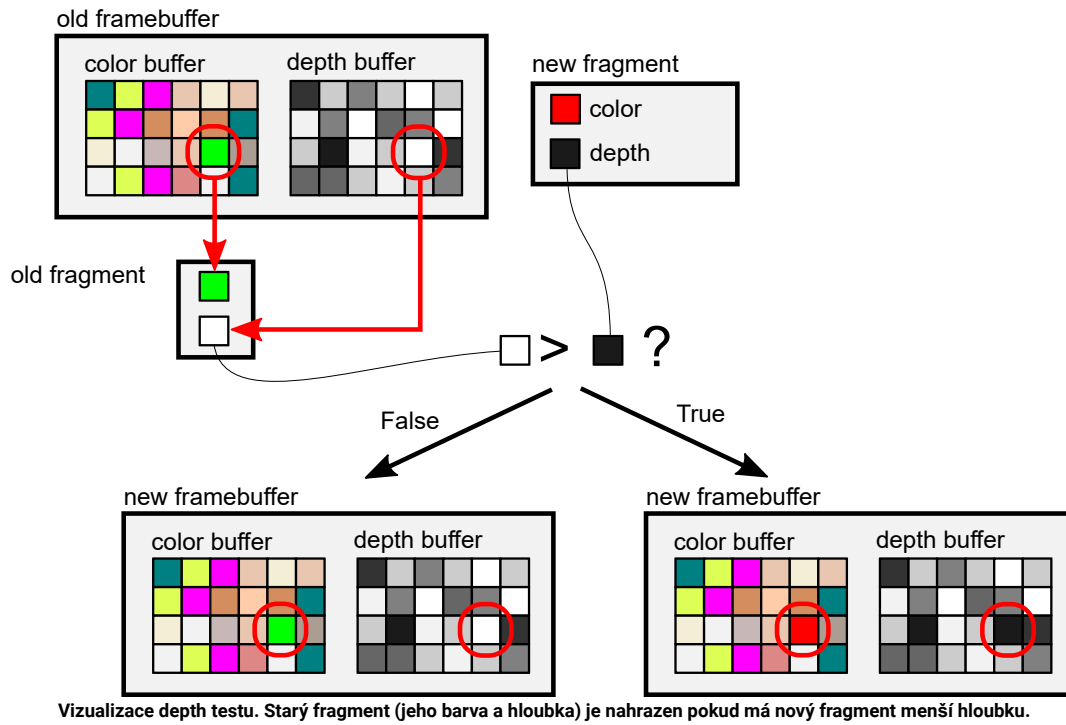
Jeho hloubkou ("z" komponenta `InFragment::gl_FragCoord` `InFragment::gl_FragCoord.z`)

A framebufferem `Frame`

```
struct Frame{
    uint8_t* color = nullptr;
    float * depth = nullptr;
    uint32_t channels = 4;
    uint32_t width = 0;
    uint32_t height = 0;
};
```

Hloubkový test

Hloubkový test je jedna z per fragment operací. Stará se o zahazování fragmentů, které jsou hlouběji než to, co už se vyrasterizovalo. Využívá k tomu hloubkový buffer. Pokud je hloubka nového fragmentu menší, je jeho barva a hloubka zapsána do framebufferu. Dejte pozor na přetečení rozsahu `OutFragment::gl_FragColor`. Před zápisem je nutné ořezat barvu do rozsahu $<0,1>$ a pak převést na bajty [0-255].



Blending

Blending je PFO operace, která místo toho, aby barvu ve framebuffer přepsala novou barvou fragmentu, tak ji smíchá. Blending má v reálu mnoho nastavení, v projektu se používá pouze alpha blending.

Fragmenty mají barvu RGBA, kde A - α je tzv. neprůhlednost.

Pokud má nový fragment $\alpha = 1$ - je absolutně neprůhledný - plně přepíše barvu ve framebufferu, když projde hloubkovým testem.

Pokud má nový fragment $\alpha = 0$ - je absolutně průhledný - vůbec barvu ve framebuffer nezmění, i když projde hloubkovým testem.

Pokud má hodnotu někde mezi, tak se barva lineárně smíchá:

$$colorBuffer_{rgb} = colorBuffer_{rgb} \cdot (1 - \alpha) + gl_FragColor_{rgb} \cdot \alpha$$

Kde $\alpha = gl_FragColor_a$

Blending + Depth modifikace

V tomto projektu je trošičku zmodifikován blending. Pokud má fragment příliš velkou průhlednost $\alpha \leq 0.5$, nebude modifikovat hloubku a nechá takovou, která tam byla. Tato modifikace v reálu obvykle neexistuje (využívá se fragment discarding), je tady jako kompromis pro zlepšení kvality vykreslování. K per fragment operacím se vážou testy 16. - 20.

16. - 20. Ověření, zda se správně fungují per fragment operace

Tyto testy ověřují, jestli se správně provádí per fragment operace a zápis do framebufferu.

```
izgProject -c --test 16
izgProject -c --test 17
izgProject -c --test 18
izgProject -c --test 19
izgProject -c --test 20
```

Upravený pseudokód funkce může vypadat takto:

```
void runVertexAssembly(){
    computeVertexID();
    readVertexAttributes();
}

void runPrimitiveAssembly(primitive,VertexArray vao,t,vertexShader,shaderInterface){
    for(every vertex v in triangle){
        InVertex inVertex;
        runVertexAssembly(inVertex,vao,t+v);
        vertexShader(primitive.vertex,inVertex,shaderInterface);
    }
}

void rasterizeTriangle(framebuffer,primitive,fragmentShader){
    for(pixels in frame){
        if(pixels in primitive){
            InFragment inFragment;
            createFragment(inFragment,primitive,barycentrics,pixelCoord,prg);
            OutFragment outFragment;
            ShaderInterface si;
            si.uniforms = ...;
            si.textures = ...;

            fragmentShader(outFragment,inFragment,si);
            clampColor(outFragment,0,1);
            perFragmentOperations(framebuffer,outFragment,inFragment.gl_FragCoord.z)
```

```

    }
}
}

void draw(GPUMemory&mem,DrawCommand const&cmd){
    for(every triangle t){
        Primitive primitive;
        runPrimitiveAssembly(primitive,vao,t,vertexShader,shaderInterface)

        runPerspectiveDivision(primitive)
        runViewportTransformation(primitive,width,height)
        rasterizeTriangle(framebuffer,primitive,fragmentShader);
    }
}

void gpu_execute(...){
}

```

Pokud to všechno budete mít hotové, mělo by vám začít fungovat zobrazování. Jestli ano, gratuluji. Máte první část hotovou.

5. Úkol - naprogramovat ořez trojúhelníků blízkou ořezovou rovinou

Tento úkol opravuje vykreslování pokud je geometrie za pozorovatelem.

Tyto úkoly můžete přeskóčit a vrátit se k nim později. Pokud se na geometrii budete dívat tak, že leží vždy před vámi, nepoznáte rozdíl.

Testy to jsou 21. - 24.

```

izgProject -c --test 21
izgProject -c --test 22
izgProject -c --test 23
izgProject -c --test 24

```

A upravený pseudokód může vypadat takto:

```

void runVertexAssembly(){
    computeVertexID()
    readVertexAttributes();
}

void runPrimitiveAssembly(primitive,vertexArray,t,vertexShader,shaderInterface){
    for(every vertex v in triangle){
        InVertex inVertex;
        runVertexAssembly(inVertex,vertexArray,t+v);
        vertexShader(primitive.vertex,inVertex,shaderInterface);
    }
}

void rasterizeTriangle(framebuffer,primitive,fragmentShader){
    for(pixel in framebuffer){
        if(pixel in primitive){
            InFragment inFragment;
            createFragment(inFragment,primitive,barycentrics,pixelCoord,vs2fs);
            OutFragment outFragment;
            fragmentShader(outFragment,inFragment,shaderInterface);
            clampColor(outFragment,0,1);
            perFragmentOperations(framebuffer,outFragment,inFragment.gl_FragCoord.z)
        }
    }
}

void draw(mem,drawCommand,gl_DrawID){
    for(every triangle t){
        Primitive primitive;
        runPrimitiveAssembly(primitive,vertexArray,t,vertexShader,gl_DrawID)

        ClippedPrimitive clipped;
        performClipping(clipped,primitive);

        for(all clipped triangle c in clipped){
            runPerspectiveDivision(c)
            runViewportTransformation(c,width,height)
            rasterizeTriangle(framebuffer,c,fragmentShader);
        }
    }
}

void clear(mem,clearCommad){
}

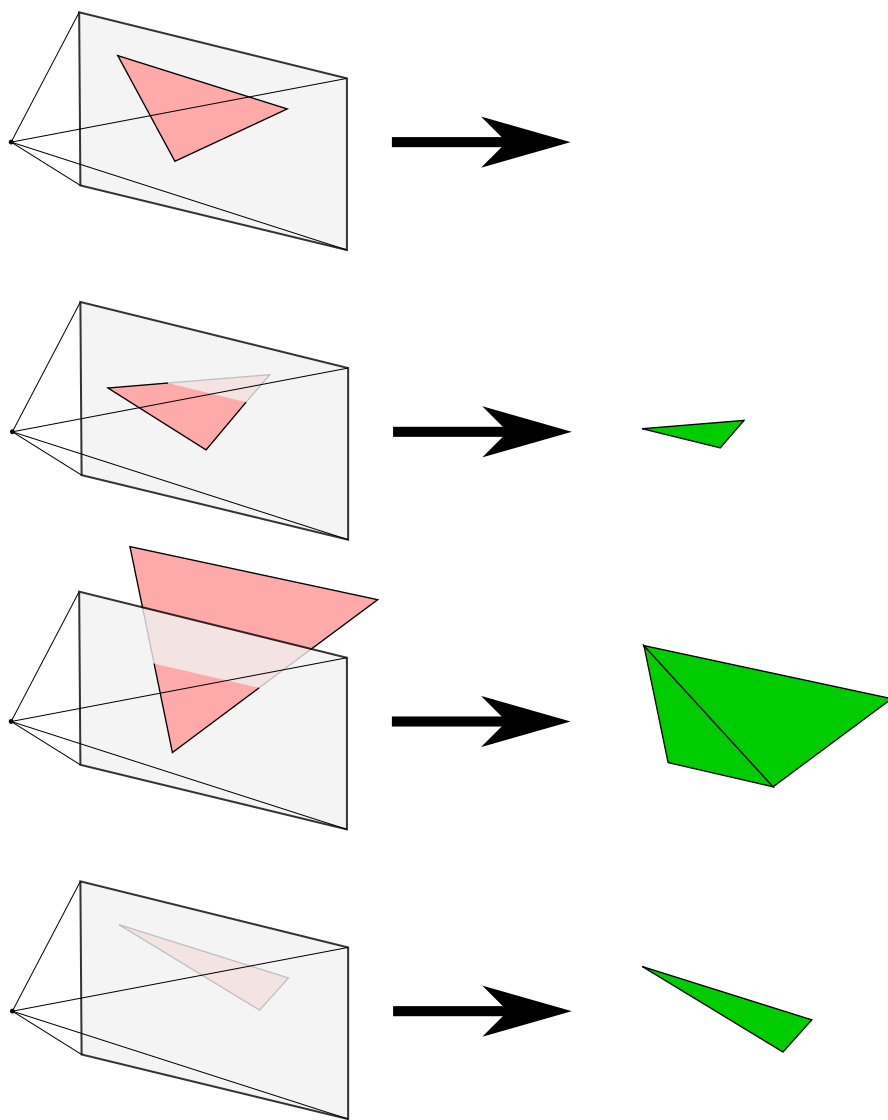
void gpu_execute(mem,commandBuffer){
    for(every command in commandBuffer){
        gl_DrawID = computeDrawID();
        if(isClearCommand)clear(mem,clearCommand)
        if(isDrawCommand )draw (mem,drawCommand,gl_DrawID)
    }
}

```

Clipping

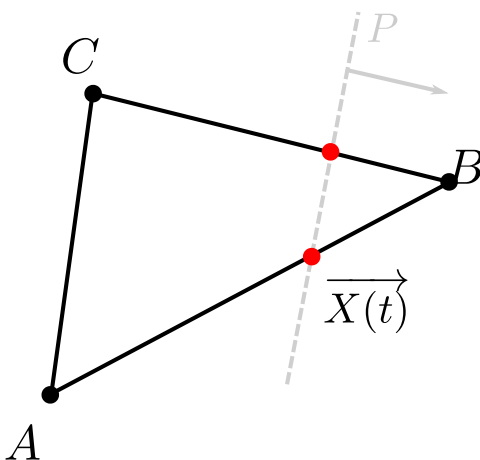
Ořez (clipping) slouží pro odstranění částí trojúhelníků, které leží mimo pohledový jehlan. Nejdůležitější je však ořez near ořezovou rovinou pohledového jehlanu. Pokud by se neprovedl ořez pomocí near roviny, pak by se vrcholy nebo i celé trojúhelníky, které leží za středem projekce promítly při perspektivním dělení na průmětnu. Ořez se provádí v clip-space - po Primitive Assembly jednotce. Pro body, které leží uvnitř pohledového tělesa platí, že jejich souřadnice splňují následující nerovnice: $-A_w \leq A_i \leq +A_w$, $i \in \{x, y, z\}$. Těchto 6 nerovnic reprezentuje jednotlivé svěny pohledového jehlanu. Nerovnice $-A_w \leq A_z$ reprezentuje podmínku pro near ořezovou rovinu.

Při ořezu trojúhelníku mohou nastat 4 případy, jsou znázorněny na následujícím obrázku:



4 varianty ořezu trojúhelníku pomocí near roviny. Počet vrcholů, které leží před ořezovou rovinou určuje typ ořezu. Při ořezu může vzniknout 0 až 2 nové trojúhelníky.

Ořez trojúhelníku pomocí near roviny lze zjednodušit na ořez hran trojúhelníku. Bod na hraně (úsečce) trojúhelníku lze vyjádřit jako: $\overrightarrow{X(t)} = \overrightarrow{A} + t \cdot (\overrightarrow{B} - \overrightarrow{A})$, $t \in [0, 1]$. $\overrightarrow{A}$, $\overrightarrow{B}$ jsou vrcholy trojúhelníka, $\overrightarrow{X(t)}$ je bod na hraně a parametr t udává posun na úsečce.



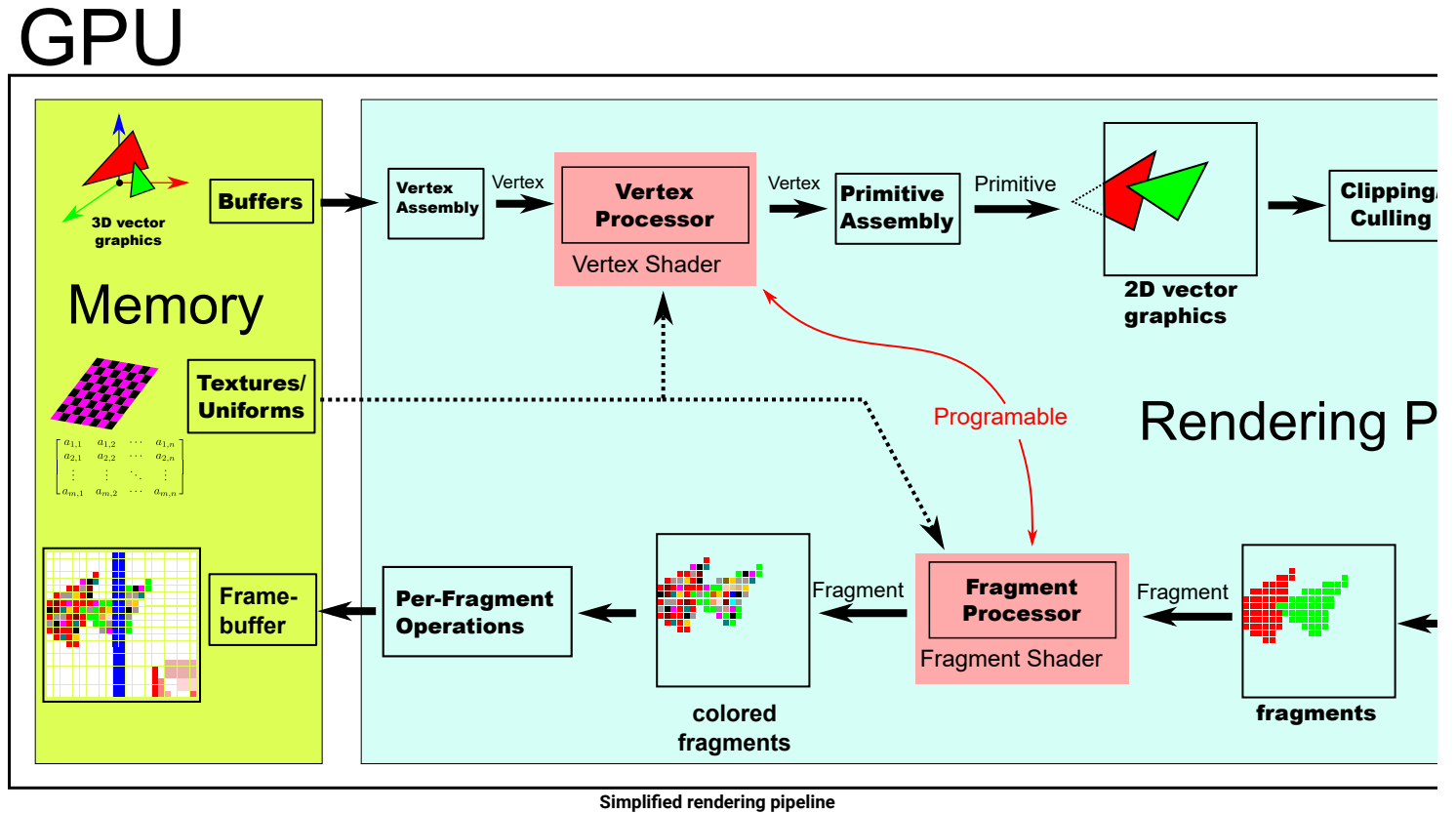
Ořez trojúhelníku pomocí ořezu hran. Při ořezu hran vzniknou nové body, ze kterých jsou následně sestaveny nové trojúhelníky.

Souřadnice bodu $\overrightarrow{X(t)}$ lze určit při vypočtení parametru t , při kterém přestane platit nerovnice pro near rovinu $-X(t)_w \leq X(t)_z$. Takové místo nastává v situaci $-X(t)_w = X(t)_z$. Po dosazení z rovnice úsečky lze vztah přepsat na:

$$\begin{aligned}
 -X(t)_w &= X(t)_z \\
 0 &= X(t)_w + X(t)_z \\
 0 &= A_w + t \cdot (B_w - A_w) + A_z + t \cdot (B_z - A_z) \\
 0 &= A_w + A_z + t \cdot (B_w - A_w + B_z - A_z) \\
 -A_w - A_z &= t \cdot (B_w - A_w + B_z - A_z) \\
 \frac{-A_w - A_z}{B_w - A_w + B_z - A_z} &= t
 \end{aligned}$$

Pozice bodu $\vec{X}(t)$ a hodnoty dalších vertex atributů lze vypočítat lineární kombinací hodnot z vrcholů úsečky pomocí parametru t následovně: $\vec{X}(t) = \vec{A} + t \cdot (\vec{B} - \vec{A})$.

Celý vykreslovací řetězec je zobrazen na následujícím obrázku:





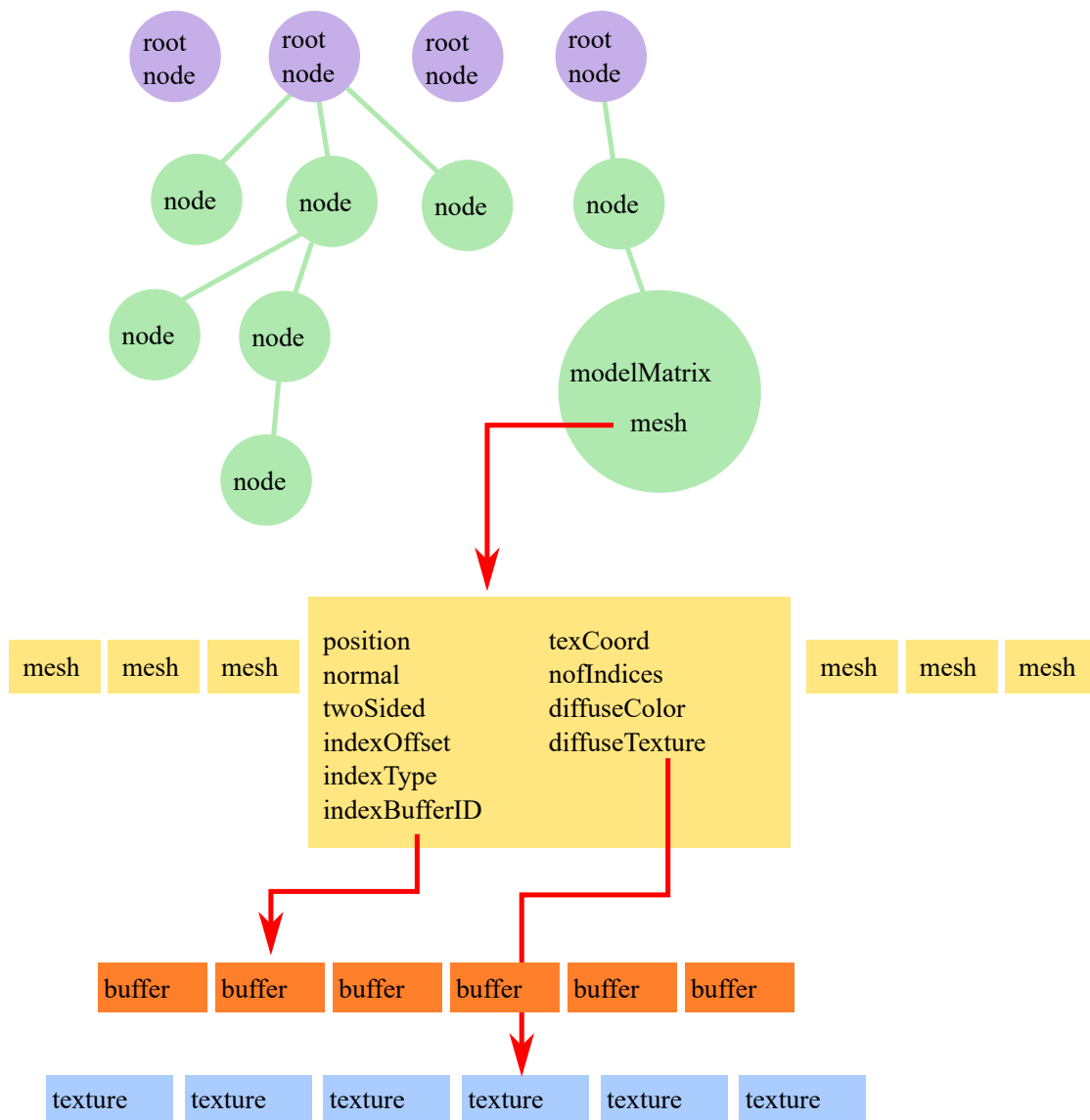
příklady

6. Úkol - Vykreslování modelů - funkce prepareModel

Tento úkol už se neváže k zobrazovacímu řetězci, ale k jeho využívání. Cílem je naprogramovat zobrazování modelů načtených ze souboru na disku. Načítání modelů už je uděláno a předpřipraveno. Vaším úkolem je jen správně vytvořit command buffer a zapsat správně data do grafické karty. Budete editovat funkci `prepareModel` v souboru `student/drawModel.cpp`. Samotné volání kreslení nebudete dělat, připravujete command buffer a paměť, které zpracuje příklad `modelMetho.cpp`.

Pro čtení z textur můžete použít funkci `read_texture`.

Struktura modelu je:



Model je složen ze 4 polí: pole kořenů, pole meshu, pole bufferů a pole textur. Kořen je uzel, který může mít potomky a může se odkazovat na mesh. Mesh obsahuje nastavení pro DrawCommand a může se odkazovat na texturu a buffery.

Vážou s k němu struktury **Model**, **Node**, **Mesh**, **Buffer**, **Texture**.

```

struct Model{
    std::vector<Mesh> meshes ;
    std::vector<Node> roots ;
    std::vector<Texture> textures;
    std::vector<Buffer> buffers ;
};

struct Node{
    glm::mat4 modelMatrix = glm::mat4(1.f);
    int32_t mesh = -1;
    std::vector<Node> children;
};

struct Mesh{
    int32_t indexBufferID = -1 ;
    size_t indexOffset = 0 ;
    IndexType indexType = IndexType::UINT32;
    VertexAttrib position ;
    VertexAttrib normal ;
    VertexAttrib texCoord ;
    uint32_t nofIndices = 0 ;
    glm::vec4 diffuseColor = glm::vec4(1.f) ;
    int diffuseTexture = -1 ;
    bool doubleSided = false ;
};

struct Buffer{
    void const* data = nullptr;
    uint64_t size = 0 ;
};

struct Texture{
    uint8_t const* data = nullptr;
    uint32_t width = 0 ;
    uint32_t height = 0 ;
    uint32_t channels = 3 ;
};

```

Pro správné vytvoření command bufferu je potřeba projít kořeny modelu a vložit všechny uzly, které mají mesh. Procházejte stromy průchodem pre order [pre order](#). Uzly se mohou odkazovat na mesh nebo nemusí (pokud je mesh=-1).

Mesh se může odkazovat na texturu nebo nemusí (pokud je diffuseTexture=-1).

V zásadě jde o to ke každému uzlu, ve kterém je odkaz na mesh, vytvořit **DrawCommand** a vložit jej do command bufferu.

Je potřeba správně spočítat modelové matice, která se budují postupný pronásobováním z kořenového uzlu.

Vytvoření command bufferu lze napsat s výhodou rekurzivně. Pseudokód možné implementace:

```
void prepareNode(GPUMemory&mem,CommandBuffer&cb,Node const&node,Model const&model,glm::mat4 const&prubeznaMatice,...){
    if(node.mesh>=0){
        mesh = model.meshes[node.mesh];

        //tvorba kreliciho prikazu
        mesh.doubleSided ...;
        mesh.position ...;
        mesh.normal ...;
        mesh.texCoord ...;
        mesh.indexBufferID ...;
        mesh.indexOffset ...;
        mesh.indexType ...
        mesh.nofVertices ...
        createDrawCall(cb)

        //zapis dat do pameti (uniformy pro draw call)
        ZKOBINUJ(prubeznaMatice,node.modelMatrix);
        inverzni tranponovana...
        mesh.diffuseColor
        mesh.doubleSided
        writeToMemory(mem);
    }

    for(size_t i=0;i<node.children.size();++i)
        prepareNode(mem,node.children[i],model,...); rekurze
}

void prepareModel(GPUMemory&mem,CommandBuffer&cb,Model const&model){
    mem.buffers = ...;
    mem.textures = ...;
    mem.programs = ...;

    clearCommand ...;

    glm::mat4 jednotkovaMatrice = glm::mat4(1.f);
    for(size_t i=0;i<model.roots.size();++i)
        prepareNode(mem,cb,model.roots[i],jednotkovaMatrice,...);
}
```

Příklad, jak připravit command buffer, můžete najít v souboru examples/phongMethod.cpp

```
void vertexShader(OutVertex&outVertex,InVertex const&inVertex,ShaderInterface const&si){
    auto const pos = glm::vec4(inVertex.attributes[0].v3,1.f);
    auto const&nor = inVertex.attributes[1].v3;
    auto const&viewMatrix = si.uniforms[0].m4;
    auto const&projectionMatrix = si.uniforms[1].m4;

    auto mvp = projectionMatrix*viewMatrix;

    outVertex.gl_Position = mvp * pos;
    outVertex.attributes[0].v3 = pos;
    outVertex.attributes[1].v3 = nor;
}

void fragmentShader(OutFragment&outFragment,InFragment const&inFragment,ShaderInterface const&si){
    auto const& light = si.uniforms[2].v3;
    auto const& cameraPosition = si.uniforms[3].v3;
    auto const& vpos = inFragment.attributes[0].v3;
    auto const& vnor = inFragment.attributes[1].v3;
    auto vvnor = glm::normalize(vnor);

    auto l = glm::normalize(light-vpos);
    float diffuseFactor = glm::dot(l, vvnor);
    if (diffuseFactor < 0.f) diffuseFactor = 0.f;

    auto v = glm::normalize(cameraPosition-vpos);
    auto r = -glm::reflect(v,vvnor);
    float specularFactor = glm::dot(r, l);
    if (specularFactor < 0.f) specularFactor = 0.f;
    float const shininess = 40.f;

    if (diffuseFactor < 0)
        specularFactor = 0;
    else
        specularFactor = powf(specularFactor, shininess);

    float t = vvnor[1];
    if(t<0.f)t=0.f;
    t*=t;
    auto materialDiffuseColor = glm::mix(glm::vec3(0.f,1.f,0.f),glm::vec3(1.f,1.f,1.f),t);

    float const nofStripes = 10;
    float factor = 1.f / nofStripes * 2.f;

    auto xs = static_cast<float>(glm::mod(vpos.x+glm::sin(vpos.y*10.f)*.1f,factor)/factor > 0.5);

    materialDiffuseColor = glm::mix(glm::mix(glm::vec3(0.f,.5f,0.f),glm::vec3(1.f,1.f,0.f),xs),glm::vec3(1.f),t);

    auto materialSpecularColor = glm::vec3(1.f);

    auto diffuseColor = materialDiffuseColor * diffuseFactor;
    auto specularColor = materialSpecularColor * specularFactor;

    auto const color = glm::min(diffuseColor + specularColor,glm::vec3(1.f));
    outFragment.gl_FragColor = glm::vec4(color,1.f);
}

Method::Method(MethodConstructionData const*){
    mem.buffers[0].data = (void const*)bunnyVertices;
    mem.buffers[0].size = sizeof(bunnyVertices);
    mem.buffers[1].data = (void const*)bunnyIndices;
    mem.buffers[1].size = sizeof(bunnyIndices);
    mem.programs[0].vertexShader = vertexShader;
```

```

mem.programs[0].fragmentShader = fragmentShader;
mem.programs[0].vs2fs[0]       = AttributeType::VEC3;
mem.programs[0].vs2fs[1]       = AttributeType::VEC3;

VertexArray vao;
vao.vertexAttrib[0].bufferID    = 0;
vao.vertexAttrib[0].type        = AttributeType::VEC3;
vao.vertexAttrib[0].stride      = sizeof(BunnyVertex);
vao.vertexAttrib[0].offset      = 0;
vao.vertexAttrib[1].bufferID    = 0;
vao.vertexAttrib[1].type        = AttributeType::VEC3;
vao.vertexAttrib[1].stride      = sizeof(BunnyVertex);
vao.vertexAttrib[1].offset      = sizeof(glm::vec3);
vao.indexBufferID = 1;
vao.indexOffset   = 0;
vao.indexType     = IndexType::UINT32;

pushClearCommand(commandBuffer,glm::vec4(.5,.5,.5,1));
pushDrawCommand (commandBuffer,sizeof(bunnyIndices)/sizeof(VertexIndex),0,vao);
}

void Method::onDraw(Frame&frame,SceneParam const&sceneParam){
    mem.framebuffer = frame;
    mem.uniforms[0].m4 = sceneParam.view ;
    mem.uniforms[1].m4 = sceneParam.proj ;
    mem.uniforms[2].v3 = sceneParam.light ;
    mem.uniforms[3].v3 = sceneParam.camera;

    gpu_execute(mem,commandBuffer);
}

```

K tomuto úkolu se vážou testy 25. až 39

```

./izgProject -c --test 25
./izgProject -c --test 26
./izgProject -c --test 27
./izgProject -c --test 28
./izgProject -c --test 29
./izgProject -c --test 30
./izgProject -c --test 31
./izgProject -c --test 32
./izgProject -c --test 33
./izgProject -c --test 34
./izgProject -c --test 35
./izgProject -c --test 36
./izgProject -c --test 37
./izgProject -c --test 38
./izgProject -c --test 39

```

Průchod modelem

Testy 25. - 31. kontrolují, jestli správně vytváříte command buffer.

Paměť

Testy 32. - 39. kontrolují, jestli správně plníte paměť grafické karty.

7. Úkol - Vykreslování modelů - vertex shader drawModel_vertexShader

Funkce **drawModel_vertexShader** reprezentuje vertex shader pro zobrazení modelů.

Jeho funkcionální spočívá v transformování vrcholů pomocí matic.

Vstupem jsou vrcholy, které mají pozici (3f), normálu (3f) a texturovací souřadnice (2f) (atributy 0, 1 a 2).

Vertex atributy **InVertex**:

- inVertex.attributes[0].v3 - pozice
- inVertex.attributes[1].v3 - normála, lokace 1, tři floaty
- inVertex.attributes[2].v2 - tex. koordináty

Výstupem jsou vrcholy, které mají pozici (3f), normálu (3f) ve world space, texturovacích souřadnic (2f) a číslo vykreslovacího příkazu (1u) (atributy 0, 1, 2, 3).

Vertex atributy **OutVertex**:

- outVertex.attributes[0].v3 - pozice
- outVertex.attributes[1].v3 - normála
- outVertex.attributes[2].v2 - tex. koordináty
- outVertex.attributes[3].u1 - číslo kreslicího příkazu (gl_DrawID)

Uniformní proměnné obsahují projectionView matici, modelovou matici, a inverzní transponovanou matici.

Uniformní proměnné Uniforms:

- si.uniforms[0].m4 - projekční view matice
- si.uniforms[10+gl_DrawID*5+0].m4 - modelová matice
- si.uniforms[10+gl_DrawID*5+1].m4 - inverzní transponovaná matice

Pozice by se měla pronásobit modelovou maticí "m*glm::vec4(pos,1.f)", aby se ztransformovala do world-space.

Normála by se měla pronásobit inverzní transponovanou modelovou maticí "itm*glm::vec4(nor,0.f)" aby se dostala do world-space.

Texturovací souřadnice se pouze přepošlou.

Pozice vrcholu gl_Position by měla být vypočtena pronásobením projectionView*model*pos.

Číslo vykreslovacího příkazu by mělo být posláno do fragment shaderu.

K tomuto úkolu se váže tests 40.

```
izgProject -c --test 40
```

8. Úkol - Vykreslování modelů - fragment shader drawMode_fragmentShader

Funkce **drawModel_fragmentShader** reprezentuje fragment shader pro zobrazení modelů.

Jeho funkcionalita spočívá v obarvování fragmentů a počítání lambertova osvětlovacího modelu.

Vstupem jsou fragmenty, které mají: pozici (3f), normálu (3f), texturovací souřadnice (2f) a číslo vykreslovacího příkazu (1u) (atributy 0,1,2,3).

Fragment Attributy **InFragment**:

- inFragment.attributes[0].v3 - pozice
- inFragment.attributes[1].v3 - normála, lokace 1, tři floaty
- inFragment.attributes[2].v2 - tex. koordináty
- inFragment.attributes[3].u1 - gl_DrawID

Výstupem je fragment s barvou a správnou průhledností α .

Uniformní proměnné obsahují pozici světla (3f), pozici kamery (3f), difuzní barvu (4f), číslo textury (1i) a příznak doubleSided (1f).

Vzhledem k tomu, že má každý mesh jinou texturu a jiné nastavení, je nutné najít správné textury podle gl_DrawID.

Uniformní proměnné Uniforms:

- si.uniforms[1].v3 - pozice světla
- si.uniforms[2].v3 - pozice kamery
- si.uniforms[10+gl_DrawID*5+2].v4 - difuzní barva
- si.uniforms[10+gl_DrawID*5+3].i1 - číslo textury nebo -1 pokud textura není
- si.uniforms[10+gl_DrawID*5+4].v1 - příznak doubleSided (1.f pokud je, 0.f pokud není)

Vstupní normálu byste měli znormalizovat $N = \text{glm::normalize}(n)$.

Difuzní barva materiálu je buď uložena v uniformní proměnné nebo v textuře.

Rozhoduje se podle toho, jestli je číslu textury záporné nebo ne.

Pokud je nastaven příznak doubleSided (je > 0), jedná se o doustraný povrch.

V takovém případě je nutné otočit normálu, pokud je otočená od kamery (využijte pozici kamery v uniformní proměnné).

Spočítejte lambertův osvětlovací model s ambientním faktorem 0.2.

K tomuto úkolu se váže test 41.

```
izgProject -c --test 41
```

9. Úkol - finální render

```
izgProject -c --test 42
```

Rozdělení souborů a složek

Projekt je rozdělen do několika podsložek:

student/ Tato složka obsahuje soubory, které využijete při implementaci projektu. Složka obsahuje soubory, které budete odevzávat a podpůrné hlavičkové soubory.

examples/ Tato složka obsahuje přiložené příklady, které využívají vámi vytvořené zobrazovací.

tests/ Tato složka obsahuje akceptační a performanční testy projektu. Akceptační testy jsou napsány s využitím knihovny catch. Testy jsou rozděleny do testovacích případů (TEST_CASE). Daný TEST_CASE testuje jednu podčást projektu.

libs/ Tato složka obsahuje pomocné knihovny **framework/** Tato složka obsahuje interní záležitosti projektu. Všechny soubory jsou napsány v C++, abyste se mohli podívat, jak to funguje.

doc/ Tato složka obsahuje doxygen dokumentaci projektu. Můžete ji přegenerovat pomocí příkazu doxygen spuštěného v root adresáři projektu.

resources/ Tato složka obsahuje modely a obrázky.

build/ Tady se čeká, že si budete sestavovat projekt, ale není to nutné, pokud víte, co děláte...

Složka student/ obsahuje soubory, které se vás přímo týkají:

gpu.cpp obsahuje definici funkce představující funkcionalitu grafické karty **gpu_execute** - tady odvedete nejvíce práce.

drawModel.cpp obsahuje definici funkce pro zpracování modelu **prepareModel** a vertex a fragment shaderu **drawModel_vertexShader** **drawModel_fragmentShader** - toto máte taky naprogramovat.

fwd.hpp obsahuje definice typů a konstanty - projděte si.

Projekt je postaven nad filozofií OpenGL/Vulkan.

Sestavení

Projekt byl testován na Ubuntu 20.04, Visual Studio 2017, 2019. Projekt vyžaduje 64 bitové sestavení. Projekt využívá build systém [CMAKE](#). CMake je program, který na základně konfiguračních souborů "CMakeLists.txt" vytvoří "makefile" v daném vývojovém prostředí. Dokáže generovat makefile pro Linux, mingw, solution file pro Microsoft Visual Studio, a další.

Postup Linux:

```
# stáhnout projekt
unzip izgProject.zip -d izgProject
cd izgProject/build
cmake ..
make -j8
./izgProject
./izgProject -h
```

Posup na Windows:

1. stáhnout projekt
2. rozbalit projekt
3. jděte do složky build/
4. ve složce build pusťte cmake-gui ..
5. pokud nevíte jak, tak pusťte cmake-gui a nastavte "Where is the source code:" na složku s projektem (obsahuje CMakeLists.txt)
6. a "Where to build the binaries: " na složku build
7. configure
8. generate
9. Otevřete vygenerovanou Microsoft Visual Studio Solution soubor.

Spouštění

Projekt je možné po úspěšném přeložení pustit přes aplikaci **izgProject**. Projekt akceptuje několik argumentů příkazové řádky, pro jejich výpis použijte parametr **-h**

- **-c** spustí akceptační testy.
- **-c -g CESTA_NEKAM/izgProject/resources/images/output.png** spustí akceptační cesty (pouze pokud jste si někdy nešikovně přesunuli soubory...)
- **-p** spustí performanční test. (vhodné až pokud aplikaci zkompilujete v RELEASE) Vyzkoušejte si

```
./izgProject -p
```

Ovládání

Aplikace se ovládá pomocí myši a klávesnice:

- stisknuté levé tlačítko myši + pohyb myši - rotace kamery
- stisknuté pravé tlačítko myši + pohyb myši - přiblížení kamery
- stisknuté prostřední tlačítko myši + pohyb myši - posun kamery do boků
- "n" - přepne na další scénu/metodu
- "p" - přepne na předcházející scénu/metodu
- "esc" - konec

Testování

Vaši implementaci si můžete ověřit sadou vestavěných akceptačních testů. Když aplikaci pustíte s parametrem "-c", pustí se akceptační testy, které ověřují funkčnost vaší implementace.

```
./izgProject -c
```

Pokud není nějaký test splněn, vypíše se k němu komentář s informacemi, co je špatně. Testy jsou seřazeny a měly by se plnit postupně. Pokud chcete pustit jeden konkrétní test (třeba 13.), pusťte aplikaci s parametry "-c --test 13".

```
./izgProject -c --test 13
```

Pokud chcete pustit všechny testy až po jeden konkrétní (třeba 5.), pusťte aplikaci s parametry "-c --up-to-test --test 5".

```
./izgProject -c --test 5 --up-to-test
```

To je užitečné, když implementujete sekci, a chcete vědět, jestli jste něco zpětně nerozbili.

Na konci výpisu testů se vám vypíše bodové hodnocení.

Odevzdávání

Odevzdejte **proj.zip**, který obsahuje jen soubory **gpu.cpp** a **drawModel.cpp**, žádné složky.

Můžete odevzdat částečné řešení, hodnotí se to, co jste odevzdali a kolik bodů vám to vypočetlo.

Před odevzdáváním si zkontrolujte, že váš projekt lze přeložit na merlinovi.

Pro ověření kompilace nemusíte na merlin kopírovat složku resources (je velká).

Pokud si chcete na merlinovi ověřit i akceptační testy stačí zkopírovat jen resources/images/output.png a resources/models/fin.glb.

Zkopírujte projekt na merlin a spusťte skript: **./merlinCompilationTest.sh**.

Odevzdávejte pouze soubory **gpu.cpp**, **drawModel.cpp**. Soubory zabalte do archivu proj.zip. Po rozbalení archivu se **NESMÍ** vytvořit žádná složka. Příkazy pro ověření na Linuxu: `zip proj.zip gpu.cpp drawModel.cpp`, `unzip proj.zip`. Studenti pracují na řešení projektu samostatně a každý odevzdá své vlastní řešení. Poradte si, ale řešení vypracujte samostatně! Žádné kopírování kódu! Nedávejte svůj projekt veřejně na github, gitlab, soureforge, pastebin, discord nebo jinam, tím se automaticky stáváte plagiátory. Neposílejte svým kamarádům kódy. Možná jim věříte, že to nekopírují, ale divili byste se. Pak byste se dostali mezi plagiátory.

Časté chyby, které nedělejte

1. student se mě nezeptá pokud neví, jak něco vyřešit. Ptejte se. Odpovím, pokud budu vědět.
2. student neodevzdá koretně zabalené soubory.
3. student si inkluduje nějaké soubory z windows, třeba windows.h - to nedělejte, překlad musí fungovat na merlinovi.
4. min, max funkce si berete odnikud - vyzkoušejte, jestli vám jde překlad na merlinovi, nebo použijte `glm::min`, `glm::max`
5. špatně pojmenovaný archiv při odevzdávání
6. soubory navíc, nebo přejmenované soubory v odevzdaném archivu
7. memory corruption, přistupujete do paměti, kam nemáte (na to je valgrind)
8. student odevzdá soubory v nějakém exotickém archivu, rar, tar.gz, 7z, iso...
9. student zkouší projekt na systému, který nebyl ověřen (ověřeno to bylo na Linuxu a Windows).
10. VirtualBox s Ubuntu je +- možný, ale může se narazit na SDL chybu no video device (asi je potřeba nainstalovat SDL: `sudo apt install xorg-dev libx11-dev libgl1-mesa-glx`).
11. Někáký problém se CMake a zprovozněním překladu na Windows (většinou je problém s cestami, zkuste dát projekt někam do jednoduché složky C:).
12. Projekt máte příliš pomalý a tak jej automatické testy předčasně utnou.

Hodnocení

Množství bodů, které dostanete, je odvozeno od množství splněných akceptačních testů a podle toho, zda vám to kreslí správně (s jistou tolerancí kvůli nepřesnosti floatové aritmetiky). Automatické opravování má k dispozici větší množství akceptačních testů (kdyby někoho napadlo je obejít). Pokud vám aplikace spadne v rámci testů, dostanete 0 bodů. Pokud aplikace nepůjde přeložit, dostanete 0 bodů.

Soutěž

Pokud váš projekt obdrží plný počet bodů, bude zařazen do soutěže o nejrychlejší implementaci zobrazovacího řetězce. Můžete přeimplementovat cokoliv v odevzdávaných souborech pokud to projde akceptačními testy a kompilací.

Spuštění měření výkonnosti:

```
./izgProject -p -f 10
```

Nejrychlejší projekty budou na věčné časy zařazeny do síně slávy. A...

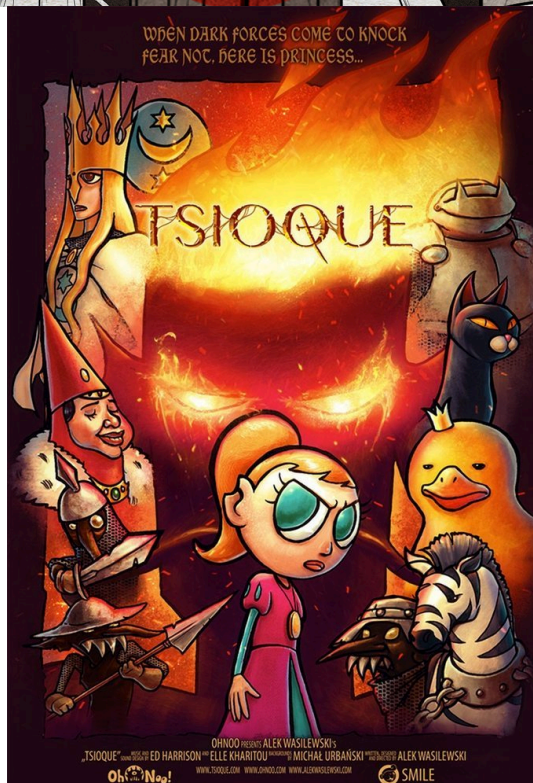
Cena za nejrychlejší projekt v roce 2020 byla:



Ceny za 1., 2. a třetí místo v nejrychlejších projektech v roce 2021 byly:



Ceny za 1., 2. a třetí místo v nejrychlejších projektu v roce 2022 byly:



Cena za nejrychlejší projekt v roce 2023 bude:

????

Závěrem

Ať se dílo daří a ať vás grafika alespoň trochu baví! V případě potřeby se nebojte zeptat (napište přímo vedoucímu projektu imilet@fit.vutbr.cz nebojte se napsat, nekoušu.
